

VIRTUAL TRY ON FASHION

A project report submitted

to

**Dr. A.P.J. ABDUL KALAM TECHNICAL
UNIVERSITY LUCKNOW**

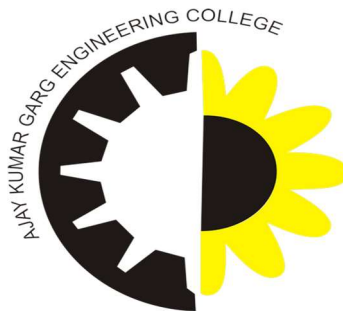
*For partial Fulfillment of the Requirement for the
Award of the Degree*

of

MASTER OF COMPUTER APPLICATIONS

by

NILESH SINGH	2300270140059,
PAWAN KUMAR PANDEY	2300270140063,
PRABHAKAR BHATT	2300270140064



**DEPARTMENT OF MASTER OF COMPUTER APPLICATIONS
AJAY KUMAR GARG ENGINEERING COLLEGE
GHAZIABAD
2024-25**

VIRTUAL TRY ON FASHION

A project report submitted

to

**Dr. A.P.J. ABDUL KALAM TECHNICAL
UNIVERSITY
LUCKNOW**

*For partial Fulfillment of the Requirement for the
Award of the Degree*

of

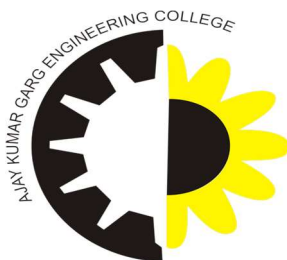
MASTER OF COMPUTER APPLICATIONS

by

NILESH SINGH	2300270140059,
PAWAN KUMAR PANDEY	2300270140063,
PRABHAKAR BHATT	2300270140064

Under the guidance of

Asst. prof : Mrs BABITA



**DEPARTMENT OF MASTER OF COMPUTER APPLICATIONS
AJAY KUMAR GARG ENGINEERING COLLEGE
GHAZIABAD
2024-25**

COLLEGE CERTIFICATE



This is to certify that project report entitled **Virtual Try On Fashion** which is submitted by **Nilesh Singh (2300270140059), Pawan Kumar Pandey (2300270140063), Prabhakar Bhatt (2300270140064)** in partial fulfilment of the requirement for the award of degree Master of Computer Applications of Dr. A.P.J. Abdul Kalam Technical University, Uttar Pradesh (AKTU), is a record of the candidates work carried out by them under my supervision. The matter embodied in this report is original and has not been submitted for the award of any other degree.

Date :

Guide Name : Mrs. Babita

Certificate of Compliance with United Nations Sustainable Development Goals

This is to certify that the project titled **Virtual Try On Fashion** submitted by **Nilesh Singh, Pawan Kumar Pandey, Prabhakar Bhatt** final year students of the Master of Computer Applications program at Ajay Kumar Garg Engineering College, Ghaziabad, have been reviewed and found to be in alignment with the following United Nations Sustainable Development Goals (SDGs). Details regarding the justification of the same are provided in Chapter 8 (Conclusion). All efforts have been made to the best of our ability and knowledge that no other SDGs are compromised or negatively impacted.

SDG No.	SDG Name	Relevance	SDG No.	SDG Name	Relevance
1	No Poverty	☐	10	Reduced Inequalities	☐
2	Zero Hunger	☐	11	Sustainable Cities and Communities	☐
3	Good Health and Well-being	☐	12	Responsible Consumption and Production	☐
4	Quality Education	☐	13	Climate Action	☐
5	Gender Equality	☐	14	Life Below Water	☐
6	Clean Water and Sanitation	☐	15	Life on Land	☐
7	Affordable and Clean Energy	☐	16	Peace, Justice, and Strong Institutions	☐
8	Decent Work and Economic Growth	☐	17	Partnerships for the Goals	☐
9	Industry, Innovation, and Infrastructure	☐			

Signature of the Students

Nilesh Singh - 2300270140059
Pawan Kumar Pandey - 2300270140063
Prabhakar Bhatt - 2300270140064

Name of the supervisor

Mrs. Babita

ACKNOWLEDGEMENT

With a profound sense of gratitude and deep respect, I express my heartfelt thanks to my guide and mentor **Ms. Babita**, Assistant Professor, MCA Department, for her valuable guidance, constant encouragement, and confidence she instilled in me throughout the development of this major project "**Virtual Try-On Fashion**." Her thorough understanding of the subject and professional guidance were indeed of immense help during all phases of the project.

I would like to extend my sincere gratitude to **Ms. Saroj Bala**, Head of the Department (HOD) of MCA, for providing the required facilities, academic support, and an inspiring environment to complete the project successfully.

This acknowledgement would remain incomplete without expressing my thanks to **Virtual Try-On Fashion**, whose concept greatly inspired and guided the core idea behind this project.

I am also deeply thankful to all the **faculty members** of the MCA Department, Ajay Kumar Garg Engineering College, for their cooperation, timely advice, and valuable time during the course of this project.

Finally, I would like to sincerely thank my parents, teachers, and friends whose continuous encouragement, valuable suggestions, and moral support have been instrumental throughout the project journey.

Name of the Students :

NILESH SINGH (2300270140059)

PAWAN KUMAR PANDEY (2300270140063)

PRABHAKAR BHATT (2300270140064)

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

AI: Artificial Intelligence

AR: Augmented Reality

CNN: Convolutional Neural Network

DFD: Data Flow Diagram

INR: Indian Rupees

SRS: Software Requirements Specification

UI: User Interface

LIST OF TABLES

Table Number & Name No.	Page
1.1: User Login Information Table	17
1.2: Product Categories Table	17
1.3: Product Details Table	18
1.4: Cart Information Table	18
1.5: Order Summary Table	18

LIST OF FIGURES

Figure Number & Name No.	Page
1.1: System Flow Chart	7
1.2: ER Diagram (Entity Relationship Diagram)	8
1.3: DFD (Data Flow Diagram)	9-10
1.4: Use Case Diagram	11
1.5: Sequence Diagram	12

ABSTRACT

This project introduces a cutting-edge **Image-Based Virtual Try-On (VTO) platform**, a revolutionary step toward transforming the landscape of online fashion retail. Combining the transformative capabilities of **Artificial Intelligence (AI)** and **Augmented Reality (AR)**, the platform enables users to virtually try on clothing items using their own photos, thereby offering a more personalized, interactive, and realistic online shopping experience.

At the core of this platform is an intelligent system that allows users to upload an image of themselves and see how different garments would appear on their actual body shape and size. Advanced AI models analyze the user's image to extract key body measurements, which are then used to dynamically adjust the virtual clothing for an accurate fit and lifelike appearance. This ensures that users not only get a visual preview of how the clothes would look, but also receive **smart sizing suggestions** tailored to their unique body dimensions. This feature directly addresses one of the most persistent challenges in online fashion: the uncertainty around correct sizing and fit.

In addition to improving user experience, this VTO platform plays a significant role in promoting **sustainability** in fashion. One of the major environmental impacts of e-commerce stems from high return rates due to sizing issues. Every returned item contributes to increased **carbon emissions, waste production, and logistical overhead**—problems that are mitigated through the implementation of accurate virtual try-ons. By reducing the frequency of returns, the platform supports more environmentally responsible shopping behavior.

Furthermore, this technology enhances **user engagement** and fosters greater **customer satisfaction**. Shoppers enjoy a more immersive and enjoyable process, one that closely mimics the physical experience of trying on clothes in a store. Brands and retailers also benefit from this approach through increased conversion rates, reduced return costs, and stronger customer loyalty.

The VTO platform acts as a **bridge between the convenience of digital shopping and the tangibility of in-person retail experiences**. It allows fashion brands to modernize their customer journey and adapt to the growing expectations of tech-savvy consumers. With its emphasis on usability, realism, and sustainability, this solution positions itself as a key player in the future of fashion technology.

In conclusion, the **Image-Based Virtual Try-On platform** is not merely a novelty but a forward-thinking solution with tangible benefits for both consumers and the environment. It redefines how people shop online—making the experience more accurate, inclusive, enjoyable, and responsible. As the fashion industry continues to evolve, innovations like this will be essential in shaping a smarter, more sustainable future.

INDEX

Content	Page No.
1. INTRODUCTION	
1.1 Background	1
1.2 Objective of Project	1
1.3 Scope of Project	1
1.4 Problem Statement	2
1.5 Literature Review	2
2. SYSTEM STUDY	
2.1 Functional Requirements	5
2.2 Non-Functional Requirements	5
2.3 User Requirements	5
2.4 System Requirements	6
2.5 Constraints and Assumptions	6
3. SYSTEM ANALYSIS	
3.1 System Flowcharts	7
3.2 E-R Diagrams	8
3.3 Data Flow Diagram (DFDs)	9
3.4 Use Case	11
3.5 Sequence Diagram	12
4. SYSTEM DESIGN	
4.1 System Architecture/Structure Chart	13
4.2 File / Database Design	13
4.3 Module Descriptions	14
4.3.1 Module 1	14
4.3.2 Module 2	14
4.4 User Interface Design	14
5. SYSTEM IMPLEMENTATION	
5.1 Development Tools	17
5.1.1 Technologies Used	17
5.1.2 Database Used	17
5.2 Coding	19
5.2.1 List of Programs	19
5.2.2 Inputs	79
5.2.3. Outputs	82

6. SYSTEM TESTING	
6.1 Preparation of Test Data & Test Oracle	84
6.2 Unit Testing, Integration Testing	84
6.3 System Testing	84
6.4 Test Cases and Results with dummy / live data	84
7. RESULTS AND ANALYSIS	
7.1 Performance Evaluation	86
7.2 Comparison with Existing Solutions	87
8. CONCLUSION AND FUTURE WORK	88
9. REFERENCES	90

CHAPTER – 1

INTRODUCTION

1.1 Background

Online fashion retail has witnessed significant growth in recent years. However, a major drawback of many online stores is the lack of a realistic “try-on” experience. Customers must rely on static images and generic size charts, which often results in misfitting clothing, high return rates, and overall customer dissatisfaction. Advances in artificial intelligence (AI), computer vision (including face and body detection), and image rendering now allow the development of virtual try-on systems. The “Virtual Try On Fashion” project leverages these technologies to create a dynamic, interactive e-commerce website that simulates a real try-on experience.

1.2 Objective of Project

The primary objectives of the Virtual Try On Fashion are as follows:

- To integrate an AI-powered virtual try-on mechanism that allows users to see how clothing fits either through uploaded images or a live camera feed.
- To implement a fully functional e-commerce platform complete with secure user registration, product browsing, “Add to Cart” and “Buy Now” functionalities—all with pricing in Indian Rupees (INR).
- To offer a dual-mode approach that supports both store-provided clothing and user-uploaded designs.
- To enhance customer satisfaction by reducing returns and providing personalized size recommendations through advanced computer vision algorithms.

1.3 Scope of Project

The project encompasses:

- A web-based application that supports both image and live camera try-on modes.
- Backend functionality using a Flask-based server integrated with a relational database (SQLite) to manage user profiles, product inventories, try-on sessions, and transactions.
- Integration of pre-trained models (e.g., StableVITON, VITON-HD) for accurate clothing overlay and fitting adjustment using AI, CNNs, and detection algorithms.
- A modern user interface (UI) that is responsive across devices and displays product information in INR.
- Full e-commerce capabilities including user management, cart operations, and secure checkout processes.

1.4 Problem Statement

Today's online fashion marketplaces offer convenience but fall short when it comes to helping customers visualize the fit and style of clothing. This results in:

- Frequent product returns due to sizing mismatches.
- Customer frustration and decreased trust in online retailers.
- Increased logistical costs and environmental waste from returned/rejected items.

The Virtual Try On Fashion addresses these issues by providing an interactive and realistic virtual try-on experience that uses advanced AI to precisely overlay clothing on user images, thereby guiding size selection and improving overall satisfaction.

1.5 Literature Review

A number of research articles and projects have explored virtual try-on systems. Key works include:

1. VTNEP: An Image-based Virtual Try-on Network with Body and Clothing Feature Preservation- The rise of e-commerce has fueled growing interest in virtual try-on (VTO) technology in fashion, reshaping how people shop online. VTO systems use artificial intelligence, computer vision, and augmented reality to let users see how clothing fits and looks in real time, creating an experience much like trying on clothes in-store. With tools like 3D body scanning, GAN-based image synthesis, and pose detection, VTO delivers realistic, personalized visuals, boosting shoppers' confidence and engagement. Recent advances in segmentation and customization make VTO even more accurate, helping tackle common issues like high return rates and customer uncertainty. This study introduces the Virtual Try-On Network (VTON), integrating advanced technologies like U2 Net and Graphonomy. By exploring how these innovations impact customer satisfaction and sales, we aim to show how VTO could shape the future of online fashion shopping, making it more interactive, satisfying, and effective.

2. VITON: An Image-based Virtual Try-on Network- The VITON paper presents an innovative approach to virtual try-on technology, focusing on a simpler, image-based method rather than intricate 3D modelling. The authors developed a two-step process: initially, they create a rough image by layering a clothing item onto a person's photo, ensuring that the individual's pose and body shape remain intact through a clothing-agnostic framework. In the second stage, a separate network refines this image, enhancing the details of the garment to produce a more realistic final result. By concentrating on 2D image synthesis, VITON effectively addresses the complexities associated with 3D virtual try-ons, making it a more user-friendly option. Tests conducted on a Zalando dataset reveal that VITON generates high-quality virtual try-on images that are not only visually appealing but also preserve the intricate features of the clothing. It surpasses existing models in performance. This research represents a noteworthy leap forward in virtual try-on technology, particularly for the online

fashion industry, offering a scalable and cost-effective solution that could significantly enhance the shopping experience for consumers and improve the capabilities of retailers.

3. 3D Web-Based Virtual Try On of Physically Simulated Clothes Nadia Magnenat-This paper introduces an innovative Virtual Try-On (VTO) application tailored for online garment fitting. This web-based tool creates a realistic and size-accurate virtual representation of users, enabling them to try on clothes in a 3D simulation environment. The system is built around three main components: body sizing, motion retargeting, and garment simulation, which work together to produce a dynamic and physically accurate model. Users can customize their garments by selecting different designs and sizes, with the simulation effectively showcasing how the clothing behaves during movement. To ensure real-time performance, the VTO employs simplified mechanical models and leverages GPU parallelization for quick garment and body deformation. The application also integrates smoothly with CAD data, potentially streamlining the garment design and production processes. This innovation promises to enhance the online shopping experience by providing an interactive and lifelike fitting experience, effectively bridging the gap between digital previews and actual physical fitting.

4. Effects of 3D Virtual “Try-on” on Online Sales and Customers’ Purchasing Experiences -This paper provides an in-depth exploration of Virtual Try-On (VTO) technology within the realm of fashion e-commerce, aiming to tackle the prevalent challenges faced by online shoppers. It introduces an innovative web-based VTO system that generates personalized 3D models based on user measurements, featuring five distinct body shapes and eleven size categories. This system stands out by transforming 2D garment images into 3D garment meshes, serving as a cost-effective alternative to traditional CAD patterns. The study's implementation with a women's wear brand yielded impressive results: a 27% reduction in return rates, a 2.24-fold increase in revisit rates, and a 2.37-fold increase in the duration of website engagement. Customer interviews highlighted six main response categories, with "confidence in apparel fit/style" emerging as the most significant, representing 32.2% of feedback.

The technology effectively combines body sizing, motion retargeting, and garment simulation to offer realistic and dynamic clothing visualization. This advancement marks a major step toward closing the gap between online and physical shopping experiences, delivering both practical advantages for retailers and enhanced shopping confidence for consumers.

5. Virtual Try-On Clothing Using Machine Learning-The image-based virtual try-on system for clothing is transforming the shopping experience by making it more convenient and personalized. This technology enables customers to see how clothes would look on them digitally, eliminating the need for physical fitting rooms and creating a more interactive online shopping environment. Although there are still some challenges to tackle, like achieving accurate fit and color representation, this innovation has already boosted user satisfaction and helped retailers present their products in a more appealing way.

Looking ahead, the future of virtual try-on technology is filled with exciting possibilities. There's great potential to enhance the realism of fabric textures and fit, and to incorporate augmented reality, which would create an even more engaging and immersive shopping experience. By focusing on these improvements, virtual try-on systems can continue to revolutionize the online retail landscape. Ultimately, this technology aims to provide consumers with a more accurate, enjoyable, and interactive way to shop, making the process not just easier but also more fun. As advancements continue, we can expect a shopping experience that feels more personal and tailored to individual preferences, changing the way we approach fashion online.

These studies have laid the groundwork for further improvements in accuracy, real-time performance, and an overall enhanced web shopping experience.

CHAPTER – 2

SYSTEM STUDY

2.1 Functional Requirements

- **User Management:**
 - Registration, login, logout, and profile management.
- **Virtual Try-On Functionality:**
 - Allow image uploads and live camera sessions.
 - Use AI for garment overlay and adjust for body measurements including face and body detection.
- **Product Management:**
 - Display product details (images, description, sizes, prices in INR).
 - Ability to add products to the cart or purchase directly (“Buy Now”).
- **User-Uploaded Clothing:**
 - Enable designers or individual users to upload their own clothing designs for try-on.

2.2 Non-Functional Requirements

- **Performance:**
 - System response should be less than 2 seconds for most interactions; try-on processing should complete within 3–5 seconds.
- **Scalability:**
 - Support for a high number of concurrent sessions (up to 10,000 users).
- **Security:**
 - HTTPS for secure communication; proper user data encryption and session management.
- **Usability:**
 - A clean, responsive interface that is intuitive and can be accessed on desktops, smartphones, and tablets.
- **Reliability:**
 - Ensure a high uptime (99.9%) and robust error handling.

2.3 User Requirements

- **Customer:**
 - A fast, secure, and interactive way to virtually try on and purchase clothes.
- **Retailer:**
 - An integrated system to reduce return rates and directly manage inventories.
- **Administrator:**
 - A backend interface to manage products, user data, and overall system health.

2.4 System Requirements

- **Software Requirements:**
 - Backend: Python with Flask, integration of AI/ML libraries (OpenCV, Mediapipe).
 - Frontend: HTML, CSS, JavaScript libraries for dynamic UI.
 - Database: SQLite (scalable to MySQL/PostgreSQL if needed).
- **Hardware Requirements:**
 - Client: Minimum dual-core processor, 4 GB RAM.
 - Server: Quad-core processor, 16 GB RAM, SSD storage for fast access.

2.5 Constraints and Assumptions

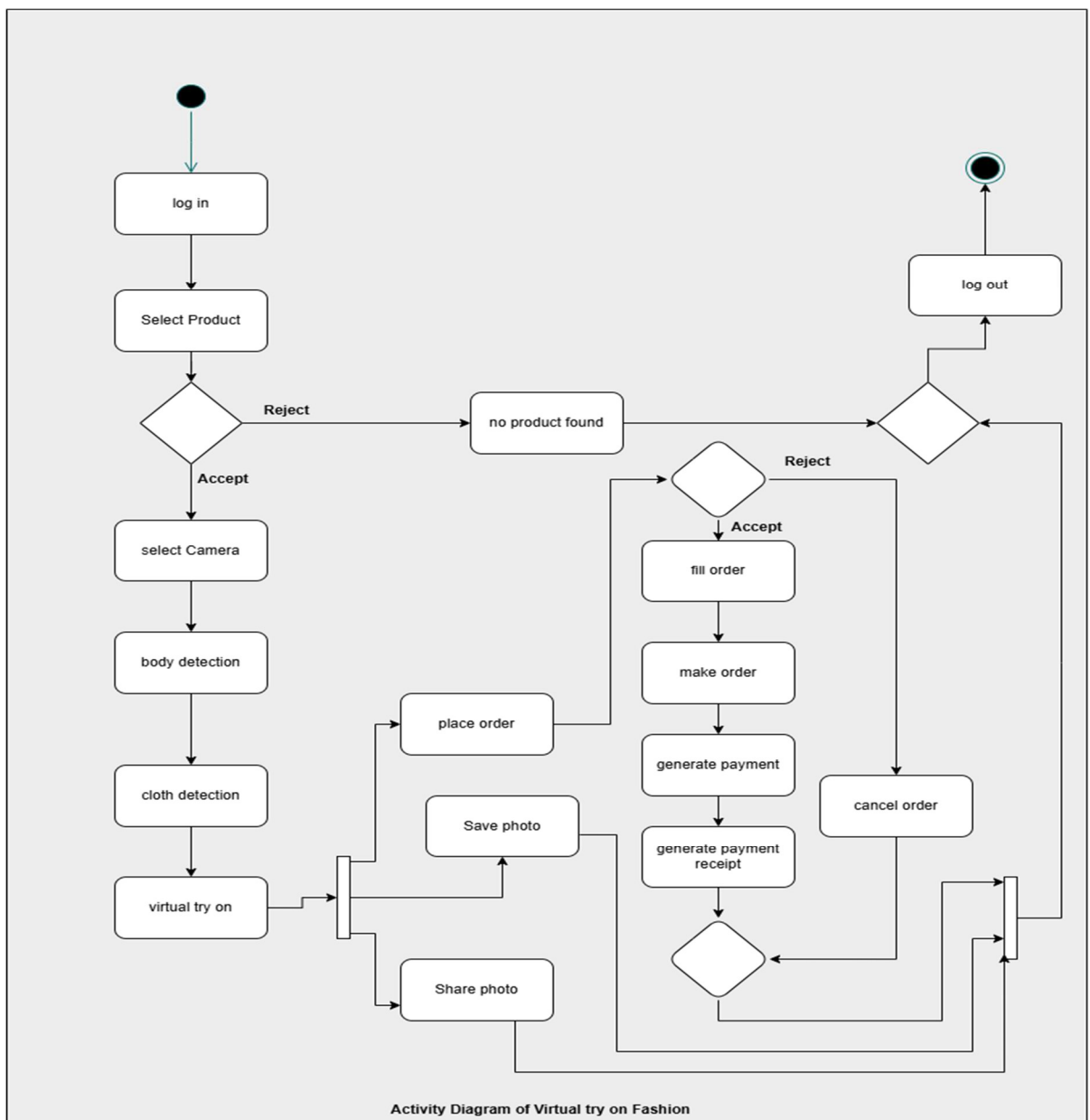
- **Constraints:**
 - Processing of high-resolution images may require significant computational resources.
 - Accuracy of try-on depends on the quality of the user's uploaded image or the device camera.
- **Assumptions:**
 - Users will have compatible devices and high-quality images.
 - The integrated pre-trained models (e.g., VITON-HD) will perform optimally under varied lighting and pose conditions.

CHAPTER – 3

SYSTEM ANALYSIS

3.1 System Flowcharts

The system flowchart begins with user registration/login followed by product browsing. After selecting a product, the user may choose to upload an image or use a live camera for try-on. The image processing module then either uses the pre-trained AI model to generate a virtual overlay or provides real-time adjustments. Once satisfied, the user proceeds with cart management and checkout.



3.2 E-R Diagrams

Key entities include:

- **User:** Attributes include UserID, Name, Email, Password, and Body Measurements.
- **Product:** Attributes include ProductID, Name, Description, Price (INR), Size Chart, Images, and Category.
- **VirtualTryOnSession:** Attributes include SessionID, UserID, ProductID, Timestamp, Uploaded Image, and Result Image.
- **Order/Cart:** Attributes include OrderID, UserID, ProductID, Status, and Payment Information.

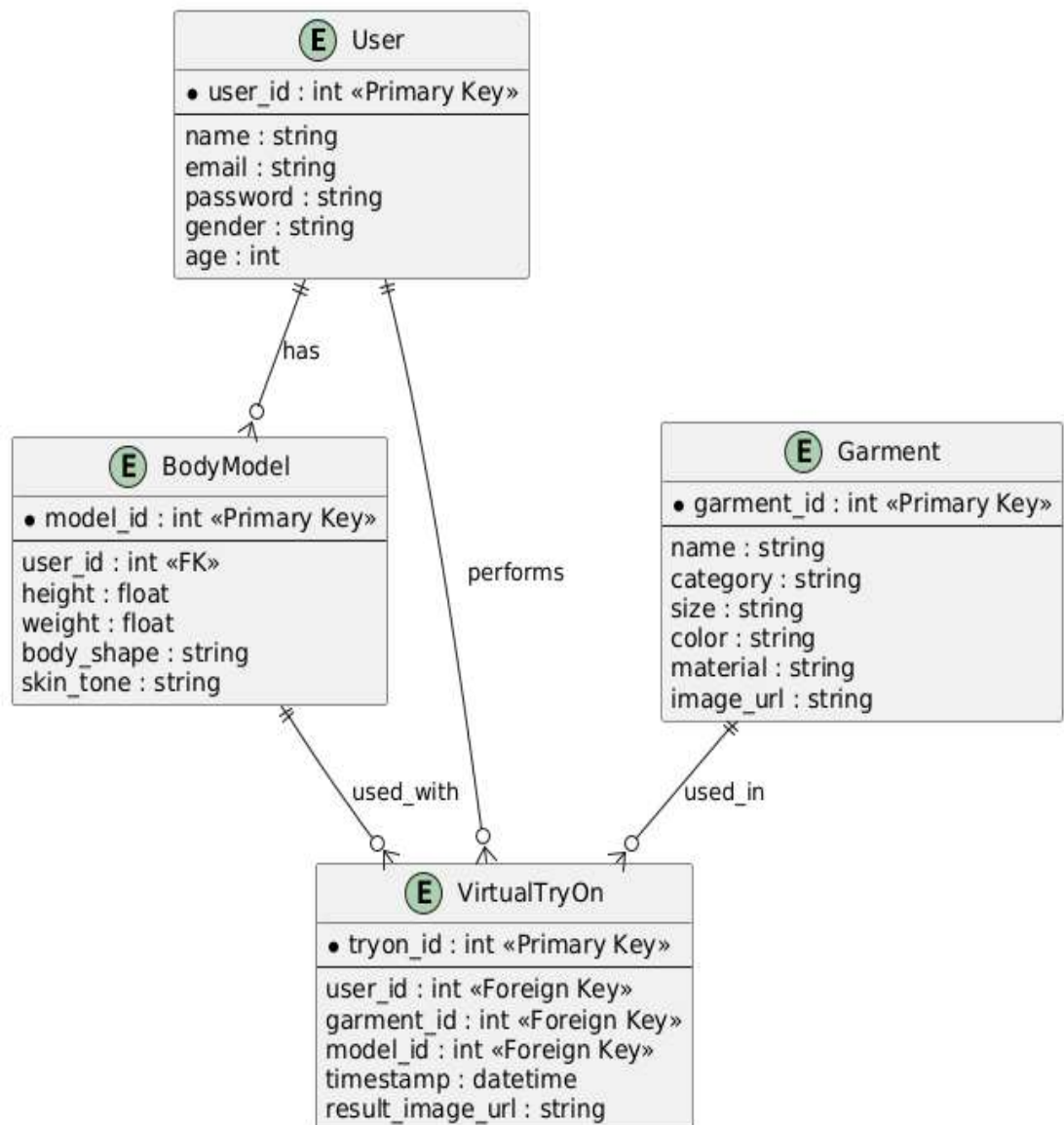


Fig 3.2 Collaboration Diagram

3.3 Data Flow Diagram (DFDs)

Level 0 :

A Level 0 DFD for a Virtual Try-On Fashion system illustrates the high-level process. It shows the user interacting with the system by uploading an image, where the system processes the image, applies clothing models, and returns a virtual try-on result. External entities include the user and the database.

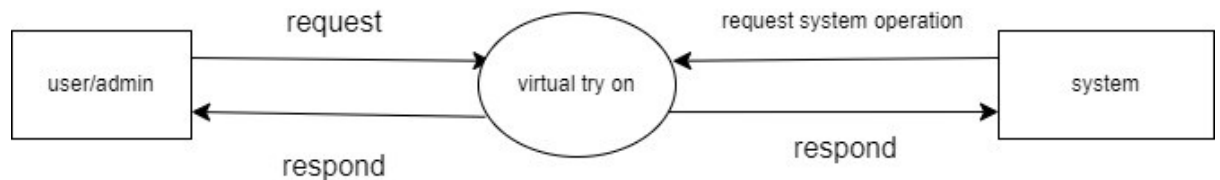


Fig 3.3.1 Level 0 Data Flow Diagram

Level 1:

A Level 1 Data Flow Diagram (DFD) for a virtual try-on fashion system outlines the main processes and data flows. It includes key elements like user input (e.g., selecting clothes), virtual try-on process (rendering the clothing on the user's image), and output (displaying the final result). Data stores may include user profiles and clothing catalogs.

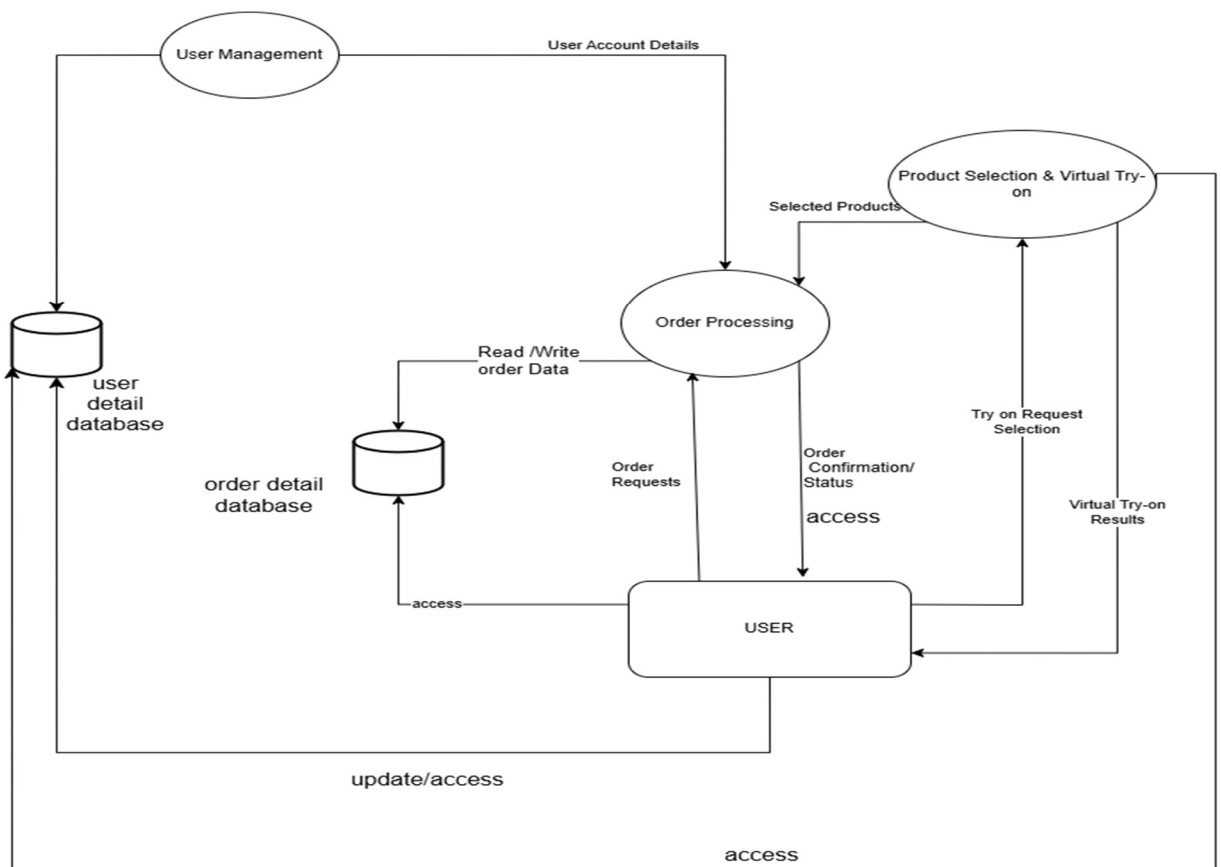


Fig 3.3.2 Level 1 Data Flow Diagram

Level 2:

A Level 2 Data Flow Diagram (DFD) for a virtual try-on fashion system depicts detailed processes, including user interaction, garment selection, and 3D model rendering. It highlights data flow between user interfaces, fashion catalogs, virtual try-on engines, and user feedback mechanisms, focusing on precise system functionality and data movement.

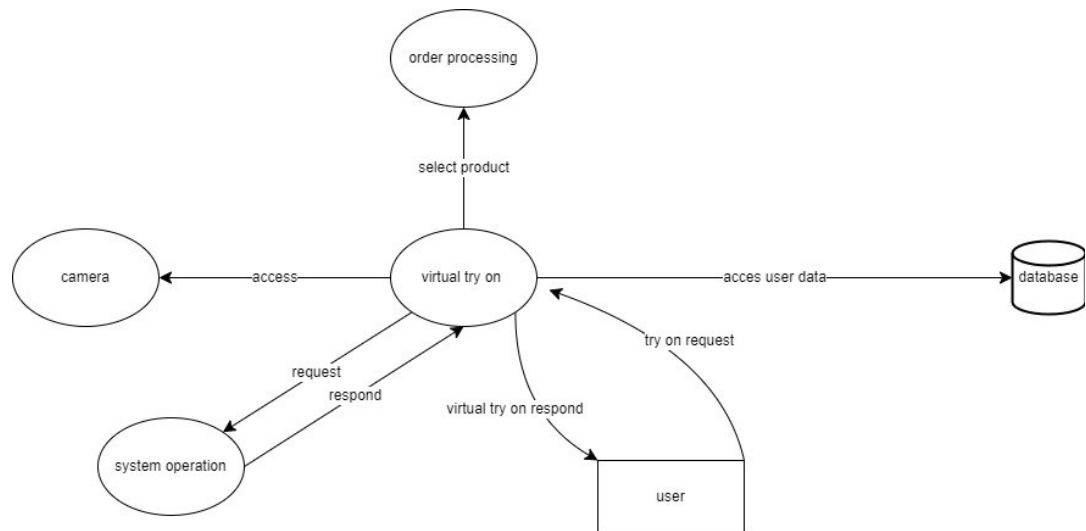


Fig 3.3.3 Level 2 Data Flow Diagram

3.4 Use Case

A use case diagram for a virtual try-on fashion system could include the following actors: "Customer," "Retailer," and "System." Key use cases include "Select Clothing," "Try On Clothing Virtually," "View Fit and Style," "Adjust Clothing Size," and "Make Purchase." The system enables users to visualize outfits and purchase them directly.

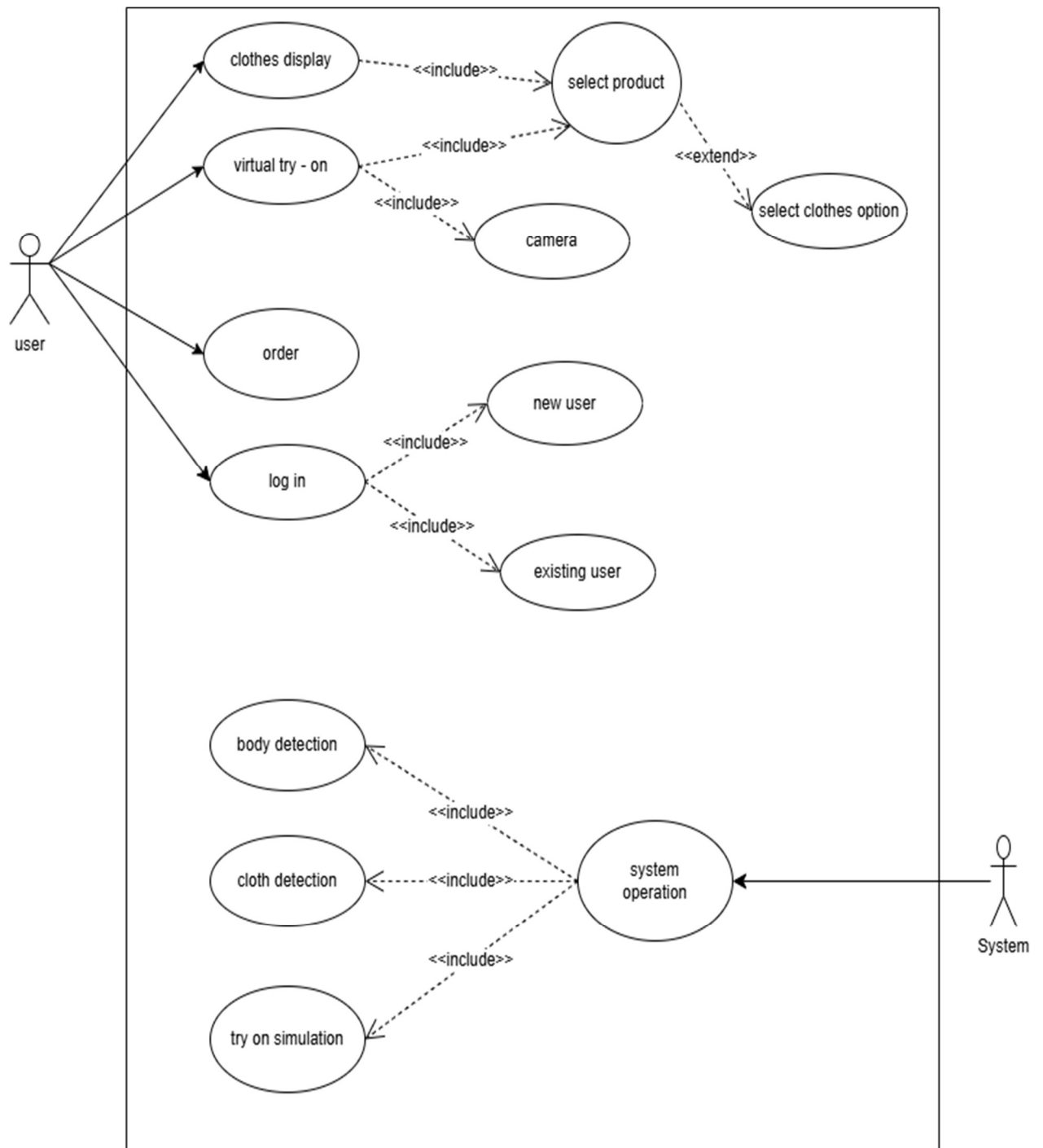


Fig 3.4 Use Case Diagram

3.5 Sequence Diagram

A sequence diagram is a type of UML (Unified Modeling Language) diagram that models the interactions between objects in a system over time. It shows how objects communicate through messages, indicating the order and timing of events. Sequence diagrams help visualize dynamic behaviour and flow of control in a system.

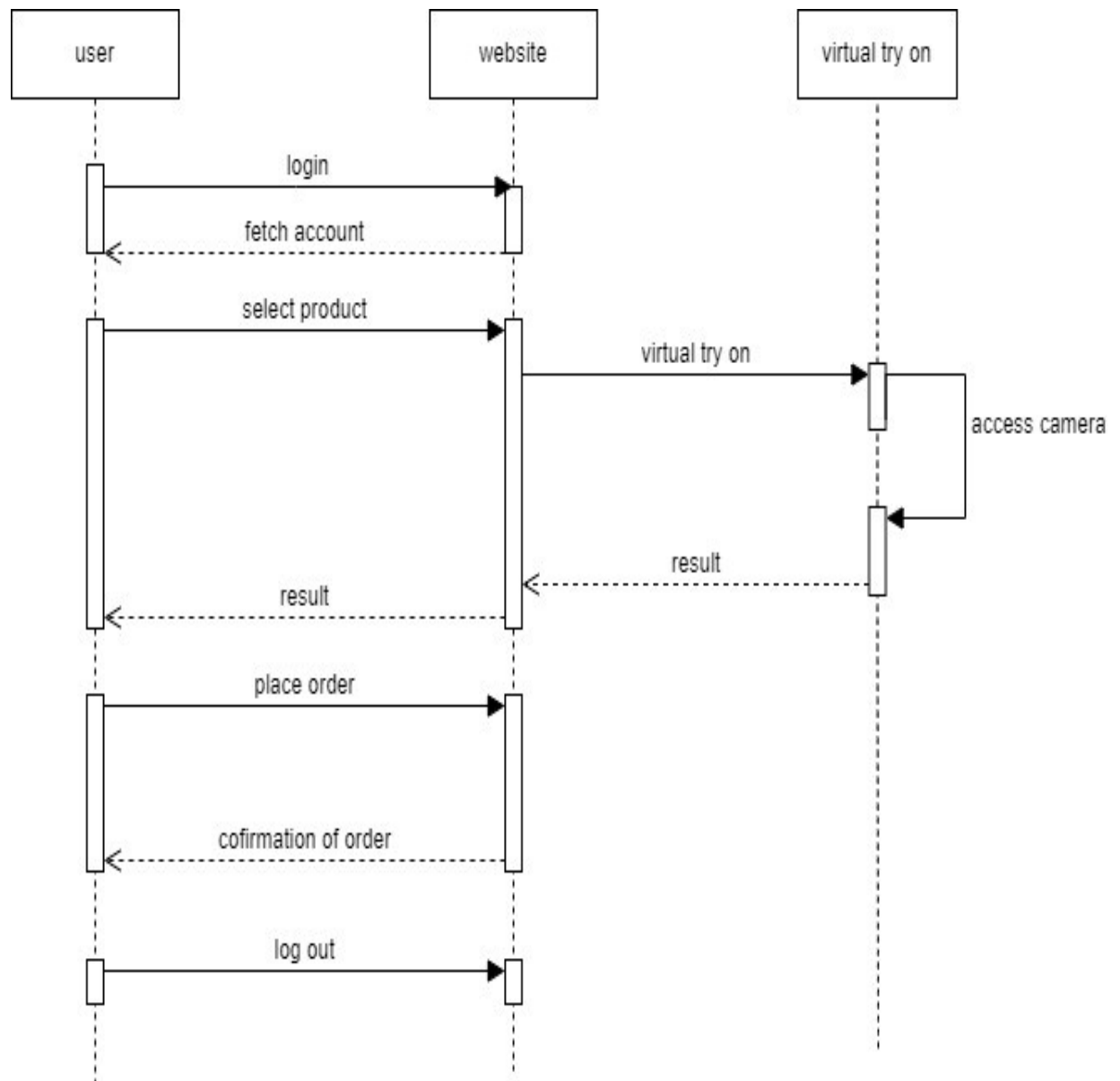


Fig 3.5 Sequence Diagram

CHAPTER - 4

SYSTEM DESIGN

4.1 System Architecture/Structure Chart

The proposed system is built using a three-tier architecture:

- **Presentation Layer:** Web-based UI using HTML, CSS, and JavaScript.
- **Business Logic Layer:** Flask/Python server handling user requests, session management, and integration with AI models.
- **Data Access Layer:** SQLAlchemy ORM interfacing with the relational database.

4.2 File / Database Design

The database schema includes the following tables:

- **Users**
- **Products**
- **VirtualTryOnSessions**
- **Orders/Carts** Each table is designed for quick data retrieval and robust transaction handling while ensuring data security and scalability.
- A Virtual Try-On database design organizes data efficiently to support seamless functionality, including user profiles, clothing items, and try-on sessions. Key entities like **Users**, **ClothingItems**, and **VirtualTryOnSessions** ensure scalability and data integrity.

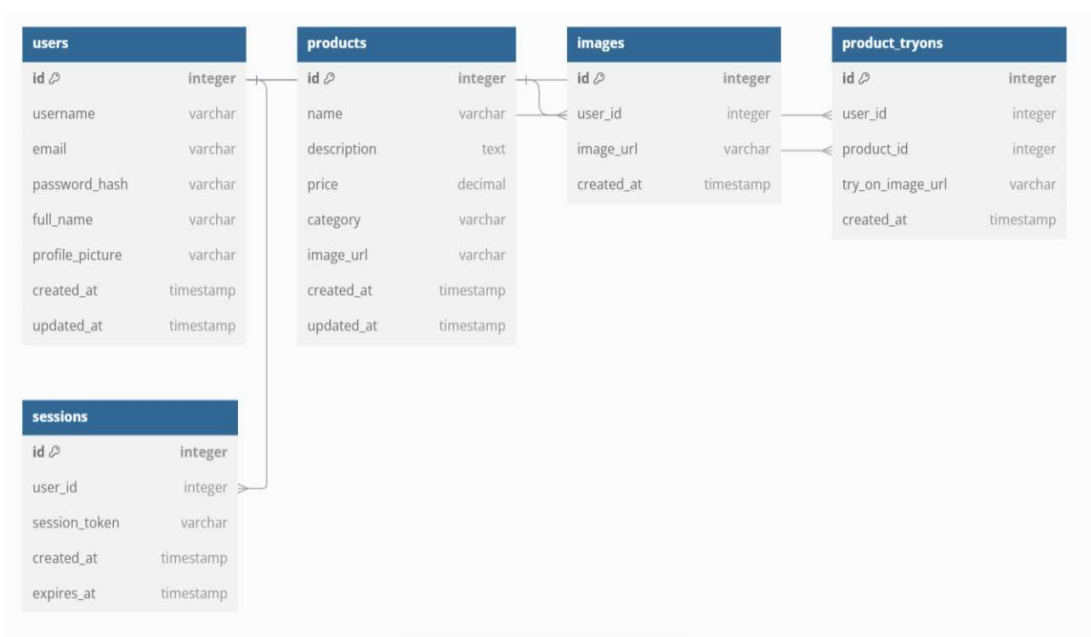


Fig 4.2 Data Base Design

4.3 Module Descriptions

4.3.1 Module 1: Virtual Try-On Processing

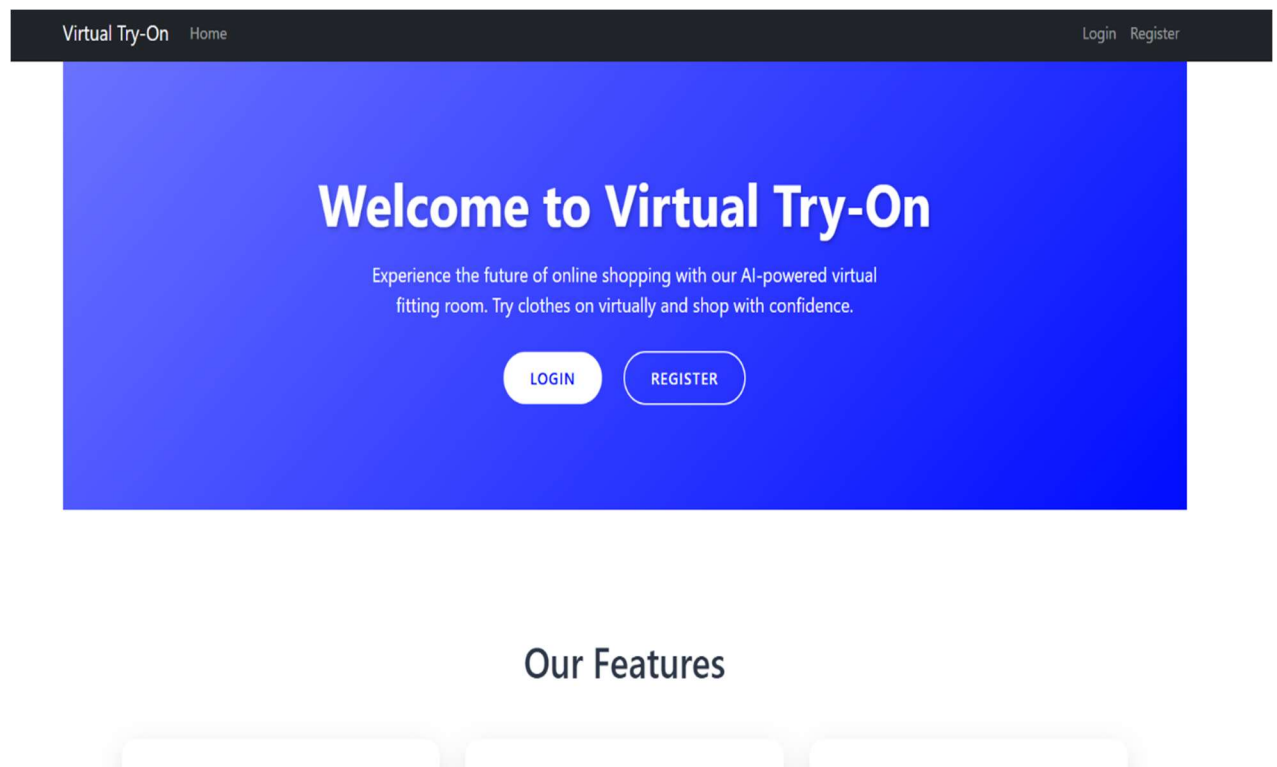
- **Functionality:**
 - Handles both uploaded images and real-time camera feeds.
 - Integrates pre-trained AI models for image overlay, employs computer vision (face and body detection), and outputs a try-on result.
- **Components:**
 - Image validation, AI processing module, and output image rendering.

4.3.2 Module 2: E-Commerce Transaction Module

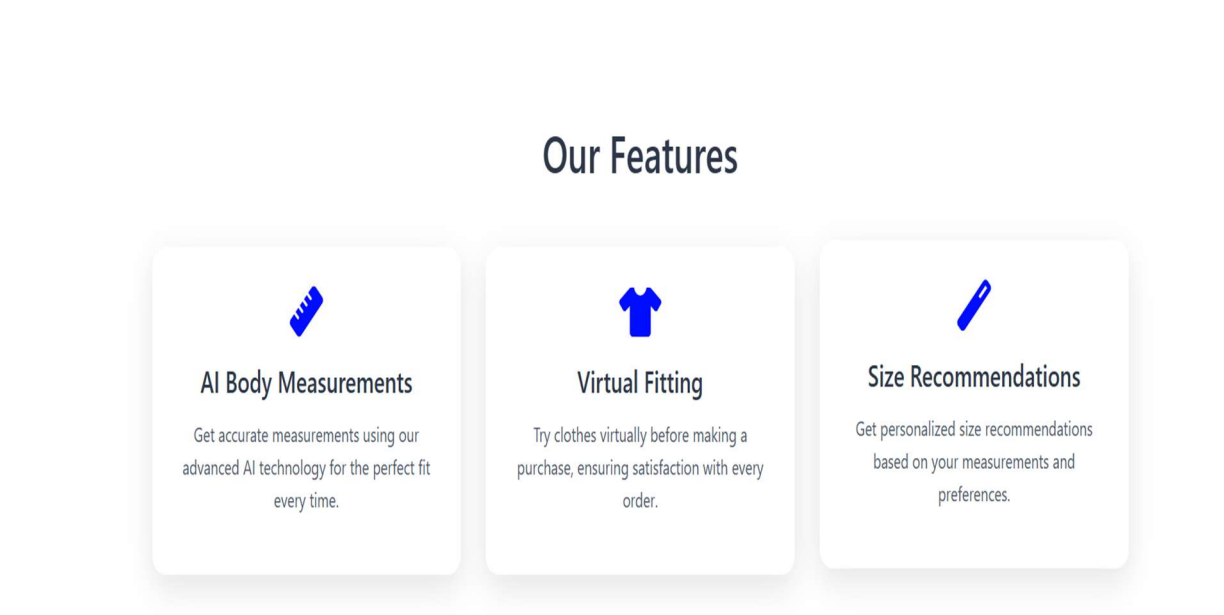
- **Functionality:**
 - Manages product presentation, shopping cart operations, and secure checkout.
 - Ensures all prices are displayed in INR and facilitates direct “Buy Now” functions.
- **Components:**
 - Product catalog, user cart management, payment gateway integration.

4.4 User Interface Design

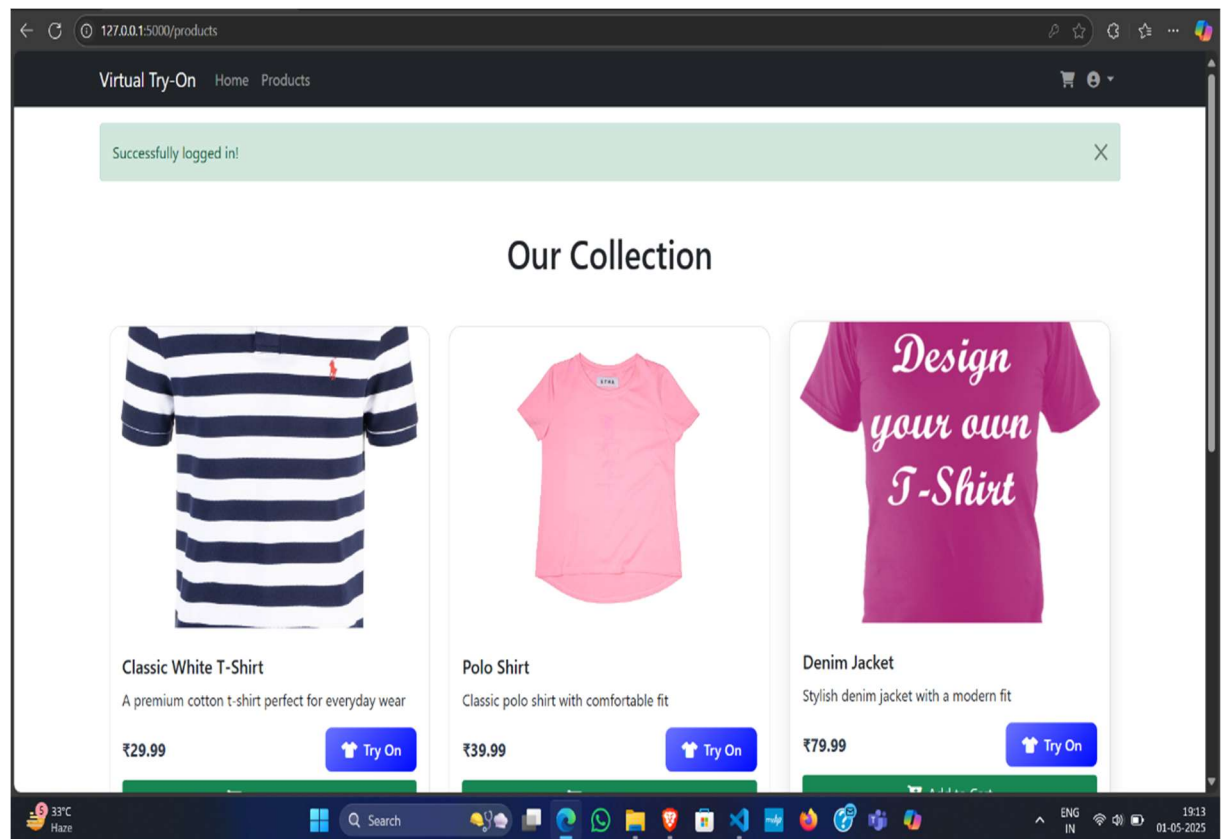
Home Page



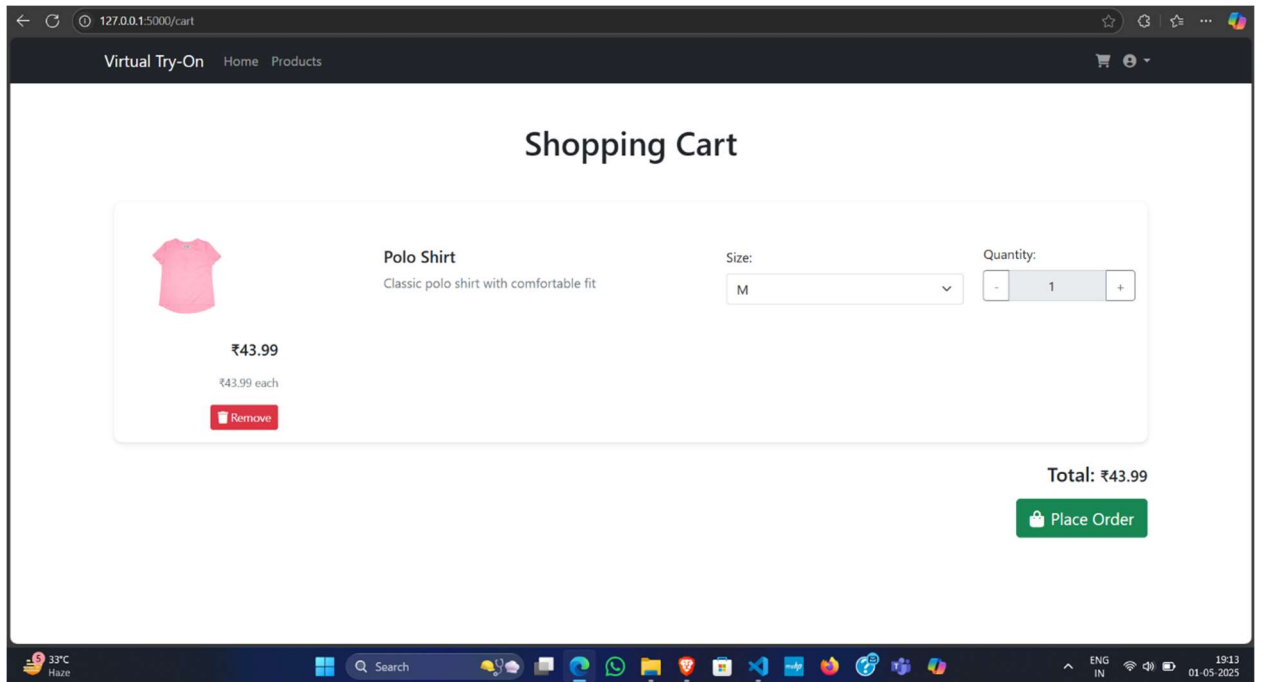
Second Page



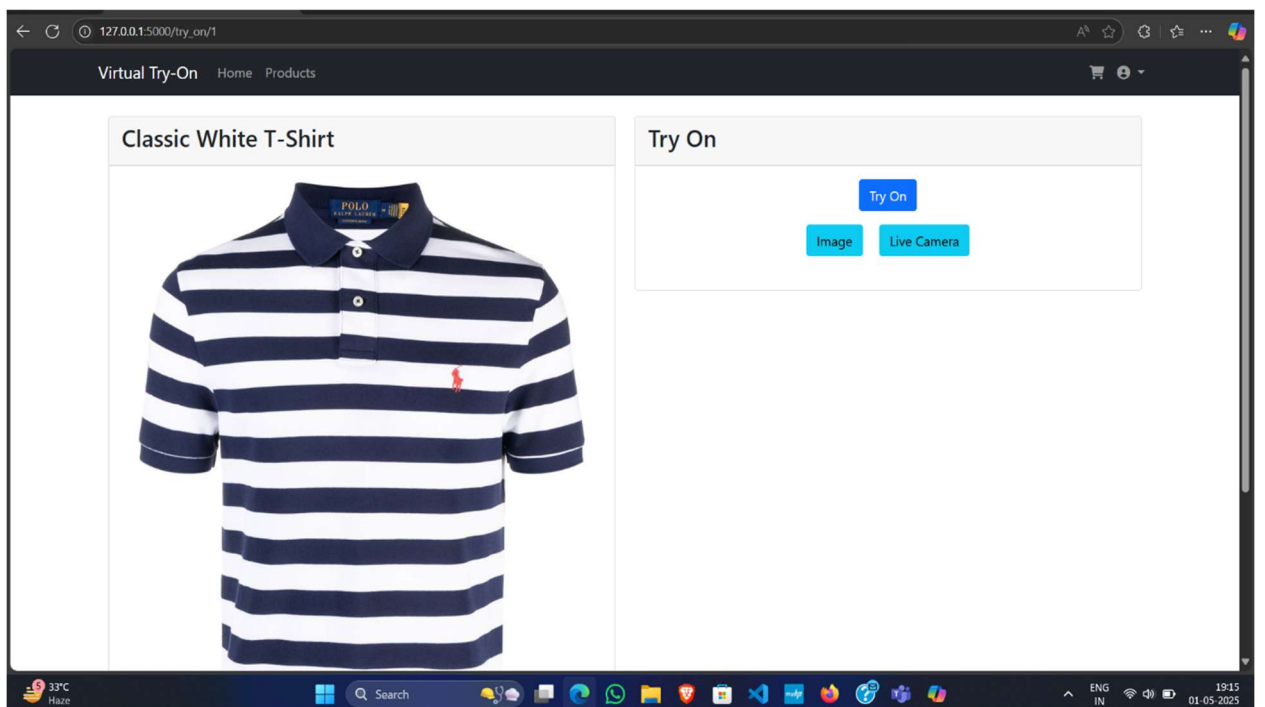
Product Page



Cart Page



Try on page



CHAPTER -5

SYSTEM IMPLEMENTATION

5.1 Development Tools

5.1.1 Technologies Used

- **Backend & Server:** Python with Flask, integration of OpenCV and Mediapipe for AI processing.
- **Frontend:** HTML5, CSS3, JavaScript (with modern frameworks/libraries as needed).

5.1.2 Database Used

- **Database:** SQLite is used initially, with options to migrate to MySQL/PostgreSQL for scale.

Table 1.1: User Login Information Table

Field Name	Data Type	Description
UserID	INT (Primary Key)	Unique identifier for each user
Name	VARCHAR(255)	Full name of the user
Email	VARCHAR(255)	Email address used for login
Password	VARCHAR(255) (Hashed)	Secure password storage
CreatedAt	TIMESTAMP	Account creation date & time

Table 1.2: Product Categories Table

CategoryID	Category Name	Description
1	Men's Clothing	Apparel for men
2	Women's Clothing	Apparel for women
3	Kids' Clothing	Apparel for children
4	Accessories	Fashion accessories

Table 1.3: Product Details Table

ProductID	Name	CategoryID	Price (INR)	Stock	Description
101	T-Shirt	1	799	50	Cotton round-neck
102	Jeans	1	1499	30	Slim fit denim jeans
201	Saree	2	2500	40	Silk saree with embroidery
301	Kids' Jacket	3	999	20	Winter jacket for children

Table 1.4: Cart Information Table

CartID	UserID	ProductID	Quantity	AddedAt
1	1001	101	2	2025-04-27 21:00
2	1002	201	1	2025-04-27 1:15

Table 1.5: Order Summary Table

OrderID	UserID	CartID	Total Amount (INR)	Status	OrderDate
5001	1001	1	1598	Pending	2025-04-27
5002	1002	2	2500	Shipped	2025-04-27

5.2 Coding

5.2.1 List of Programs

- **app.py**

```
import os
from flask import Flask, render_template, request, redirect, url_for, flash, jsonify, session,
send_from_directory
from flask_sqlalchemy import SQLAlchemy
from werkzeug.utils import secure_filename
from models.product import Product, db
from utils.tryon_utils import EnhancedVirtualTryOn
from utils.body_detection import BodyDetector
from werkzeug.security import generate_password_hash, check_password_hash
import cv2
import numpy as np
import mediapipe as mp
from datetime import datetime
import jwt
import secrets
import time
import sys
from utils.model_processor import ClothingModelProcessor
import json
from utils.pose_processor import PoseProcessor
from utils.clothing_adjuster import ClothingAdjuster
from utils.surface_visualizer import create_visualization
from routes.webrtc_routes import webrtc
import base64

# Windows console encoding fix
if sys.platform.startswith('win'):
    import locale
    if sys.stdout.encoding != 'utf-8':
        sys.stdout.reconfigure(encoding='utf-8')

app = Flask(__name__)
app.config['SECRET_KEY'] = secrets.token_hex(16)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///virtualtry.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.config['UPLOAD_FOLDER'] = os.path.join(app.root_path, 'static', 'uploads')

# Configure allowed file extensions
ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif', 'obj', 'stl'}

def allowed_file(filename):
    return '.' in filename and \
        filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
```

```

# Initialize database
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password = db.Column(db.String(200), nullable=False)

class Product(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    description = db.Column(db.Text)
    price = db.Column(db.Float, nullable=False)
    size_chart = db.Column(db.JSON)
    image_url = db.Column(db.String(200))
    category = db.Column(db.String(50))
    try_on_enabled = db.Column(db.Boolean, default=True)

class Cart(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=False)
    product_id = db.Column(db.Integer, db.ForeignKey('product.id'), nullable=False)
    quantity = db.Column(db.Integer, default=1)
    size = db.Column(db.String(10))
    price = db.Column(db.Float, nullable=False)

    user = db.relationship('User', backref=db.backref('cart_items', lazy=True))
    product = db.relationship('Product')

class Order(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=False)
    total_amount = db.Column(db.Float, nullable=False)
    created_at = db.Column(db.DateTime, nullable=False)
    shipping_name = db.Column(db.String(100), nullable=False)
    shipping_email = db.Column(db.String(120), nullable=False)
    shipping_phone = db.Column(db.String(20), nullable=False)
    shipping_address = db.Column(db.Text, nullable=False)
    shipping_city = db.Column(db.String(100), nullable=False)
    shipping_state = db.Column(db.String(100), nullable=False)
    shipping_pincode = db.Column(db.String(10), nullable=False)
    payment_method = db.Column(db.String(20), nullable=False)
    payment_status = db.Column(db.String(20), default='pending')
    user = db.relationship('User', backref=db.backref('orders', lazy=True))
    items = db.relationship('OrderItem', backref='order', lazy=True)

```

```

class OrderItem(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    order_id = db.Column(db.Integer, db.ForeignKey('order.id'), nullable=False)
    product_id = db.Column(db.Integer, db.ForeignKey('product.id'), nullable=False)
    quantity = db.Column(db.Integer, nullable=False)
    size = db.Column(db.String(10), nullable=False)
    price = db.Column(db.Float, nullable=False)

    product = db.relationship('Product')
db.init_app(app)

def init_db():
    with app.app_context():
        try:
            # Drop all tables and recreate them
            db.drop_all()
            db.create_all()

            # Create products directory if it doesn't exist
            products_dir = os.path.join(app.root_path, 'static', 'products')
            os.makedirs(products_dir, exist_ok=True)

            # Create uploads directory if it doesn't exist
            uploads_dir = os.path.join(app.root_path, 'static', 'uploads')
            os.makedirs(uploads_dir, exist_ok=True)

            # Check if we already have any products
            if not Product.query.first():
                # Add sample products
                sample_products = [
                    Product(
                        name='Classic White T-Shirt',
                        description='A premium cotton t-shirt perfect for everyday wear',
                        price=29.99,
                        size_chart={'S': [36, 28], 'M': [38, 30], 'L': [40, 32], 'XL': [42, 34]},
                        image_url='products/19620598_44329692_1000.jpg',
                        category='T-Shirts',
                        try_on_enabled=True
                    ),
                    Product(
                        name='Polo Shirt',
                        description='Classic polo shirt with comfortable fit',
                        price=39.99,
                        size_chart={'S': [36, 28], 'M': [38, 30], 'L': [40, 32], 'XL': [42, 34]},
                        image_url='products/2.png',
                        category='T-Shirts',
                        try_on_enabled=True
                    ),

```

```

        Product(
            name='Denim Jacket',
            description='Stylish denim jacket with a modern fit',
            price=79.99,
            size_chart={'S': [36, 24], 'M': [38, 25], 'L': [40, 26], 'XL': [42, 27]},
            image_url='products/7.png',
            category='Jackets',
            try_on_enabled=True
        ),
        Product(
            name='Formal Black Blazer',
            description='Elegant black blazer for formal occasions',
            price=129.99,
            size_chart={'S': [36, 28], 'M': [38, 29], 'L': [40, 30], 'XL': [42, 31]},
            image_url='products/R.jpeg',
            category='Formal Wear',
            try_on_enabled=True
        ),
        Product(
            name='Casual Hoodie',
            description='Comfortable hoodie for casual wear',
            price=49.99,
            size_chart={'S': [36, 26], 'M': [38, 27], 'L': [40, 28], 'XL': [42, 29]},
            image_url='products/6.png',
            category='Casual Wear',
            try_on_enabled=True
        )
    ]

    for product in sample_products:
        db.session.add(product)

    # Check if we already have any users
    if not User.query.first():
        # Create a test user
        test_user = User(
            name='Test User',
            email='test@example.com',
            password=generate_password_hash('password123')
        )
        db.session.add(test_user)

    # Commit all changes
    db.session.commit()
    print("Database initialized successfully!")

except Exception as e:
    print(f'Error initializing database: {str(e)}')
    db.session.rollback()

```

```

# Initialize body detector
body_detector = BodyDetector()

# Initialize processors
pose_processor = PoseProcessor()
clothing_adjuster = ClothingAdjuster()

# Register WebRTC routes
app.register_blueprint(webrtc, url_prefix='/webrtc')

def price_display(amount):
    # You can modify this function to use different currency symbols and formats
    currency_symbol = '₹' # Indian Rupee symbol
    return f'{currency_symbol} {amount:,.2f}'

@app.context_processor
def utility_processor():
    return dict(price_display=price_display)

@app.route('/')
def home():
    return render_template('index.html')

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        try:
            email = request.form.get('email')
            password = request.form.get('password')

            if not email or not password:
                flash('Please provide both email and password', 'error')
                return render_template('login.html')

            user = User.query.filter_by(email=email).first()
            if user and check_password_hash(user.password, password):
                session['user_id'] = user.id
                flash('Successfully logged in!', 'success')
                return redirect(url_for('products'))

            flash('Invalid email or password', 'error')
            return render_template('login.html')
        except Exception as e:
            app.logger.error(f'Login error: {str(e)}')
            flash('An error occurred during login', 'error')
            return render_template('login.html')

    return render_template('login.html')

```

```

@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        try:
            name = request.form.get('name')
            email = request.form.get('email')
            password = request.form.get('password')

            if not all([name, email, password]):
                flash('Please fill in all fields', 'error')
                return render_template('register.html')

            if User.query.filter_by(email=email).first():
                flash('Email already registered', 'error')
                return render_template('register.html')

            user = User(
                name=name,
                email=email,
                password=generate_password_hash(password)
            )
            db.session.add(user)
            db.session.commit()

            flash('Registration successful! Please login.', 'success')
            return redirect(url_for('login'))
        except Exception as e:
            db.session.rollback()
            app.logger.error(f'Registration error: {str(e)}')
            flash('An error occurred during registration', 'error')
            return render_template('register.html')

    return render_template('register.html')

@app.route('/logout')
def logout():
    session.pop('user_id', None)
    flash('Successfully logged out!', 'success')
    return redirect(url_for('home'))

@app.route('/products')
def products():
    if 'user_id' not in session:
        flash('Please login to view products', 'error')
        return redirect(url_for('login'))
    products = Product.query.all()
    return render_template('products.html', products=products)

```

```

@app.route('/cart')
def cart():
    if 'user_id' not in session:
        return redirect(url_for('login'))

    cart_items = []
    total_price = 0

    # Get cart items from database
    db_cart_items = Cart.query.filter_by(user_id=session['user_id']).all()

    for cart_item in db_cart_items:
        product = cart_item.product
        if product:
            # Get size information from the product's size chart
            sizes = list(product.size_chart.keys()) if product.size_chart else ['S', 'M', 'L', 'XL']

            cart_items.append({
                'id': product.id,
                'name': product.name,
                'description': product.description,
                'base_price': product.price,
                'unit_price': cart_item.price,
                'unit_price_display': price_display(cart_item.price),
                'price': cart_item.price * cart_item.quantity,
                'price_display': price_display(cart_item.price * cart_item.quantity),
                'image_url': product.image_url,
                'sizes': sizes,
                'selected_size': cart_item.size,
                'quantity': cart_item.quantity
            })
            total_price += cart_item.price * cart_item.quantity

    return render_template('cart.html',
                           cart_items=cart_items,
                           total_price_display=price_display(total_price))

@app.route('/add_to_cart', methods=['POST'])
def add_to_cart():
    if 'user_id' not in session:
        return jsonify({'error': 'Please login first'}), 401

    product_id = request.form.get('product_id')
    if not product_id:
        return jsonify({'error': 'Product ID is required'}), 400

    # Check if product exists
    product = Product.query.get(product_id)
    if not product:

```

```

        return jsonify({'error': 'Product not found'}), 404

    # Check if product is already in cart
    existing_item = Cart.query.filter_by(
        user_id=session['user_id'],
        product_id=product_id
    ).first()

    if existing_item:
        return jsonify({'success': True, 'message': 'Product is already in cart'})

    # Add new item to cart
    # Default to size 'M' and calculate price with size multiplier
    size = 'M'
    size_multiplier = 1.1 # M size multiplier
    price = product.price * size_multiplier

    cart_item = Cart(
        user_id=session['user_id'],
        product_id=product_id,
        quantity=1,
        size=size,
        price=price
    )

    try:
        db.session.add(cart_item)
        db.session.commit()
        return jsonify({
            'success': True,
            'message': 'Product added to cart',
            'cart_item': {
                'product_name': product.name,
                'quantity': 1,
                'size': size,
                'price': price,
                'price_display': price_display(price)
            }
        })
    except Exception as e:
        db.session.rollback()
        app.logger.error(f'Error adding to cart: {str(e)}')
        return jsonify({'error': 'Error adding product to cart'}), 500

@app.route('/remove_from_cart', methods=['POST'])
def remove_from_cart():
    if 'user_id' not in session:
        return redirect(url_for('login'))

```

```

product_id = request.form.get('product_id')
if product_id:
    cart_item = Cart.query.filter_by(
        user_id=session['user_id'],
        product_id=product_id
    ).first()

    if cart_item:
        try:
            db.session.delete(cart_item)
            db.session.commit()
            flash('Product removed from cart!', 'success')
        except Exception as e:
            db.session.rollback()
            app.logger.error(f'Error removing from cart: {str(e)}')
            flash('Error removing product from cart!', 'error')

    return redirect(url_for('cart'))

@app.route('/update_size', methods=['POST'])
def update_size():
    if 'user_id' not in session:
        return jsonify({'error': 'Not logged in'}), 401

    product_id = request.form.get('product_id')
    size = request.form.get('size')

    if product_id and size:
        session[f'size_{product_id}'] = size
        session.modified = True

        # Calculate new price
        product = Product.query.get(product_id)
        if product:
            size_multipliers = {'S': 1.0, 'M': 1.1, 'L': 1.2, 'XL': 1.3}
            new_price = product.price * size_multipliers.get(size, 1.0)
            return jsonify({
                'success': True,
                'new_price': new_price,
                'price_display': price_display(new_price)
            })

    return jsonify({'error': 'Invalid request'}), 400

@app.route('/update_quantity', methods=['POST'])
def update_quantity():
    if 'user_id' not in session:
        return jsonify({'error': 'Not logged in'}), 401

```

```

product_id = request.form.get('product_id')
quantity = request.form.get('quantity')

if product_id and quantity:
    try:
        quantity = int(quantity)
        if 1 <= quantity <= 10: # Validate quantity range
            # Get cart item
            cart_item = Cart.query.filter_by(
                user_id=session['user_id'],
                product_id=product_id
            ).first()

            if cart_item:
                cart_item.quantity = quantity
                db.session.commit()

                # Calculate new price
                total_price = cart_item.price * quantity

                return jsonify({
                    'success': True,
                    'unit_price': cart_item.price,
                    'unit_price_display': price_display(cart_item.price),
                    'total_price': total_price,
                    'total_price_display': price_display(total_price)
                })

            return jsonify({'error': 'Item not found in cart'}), 404

        return jsonify({'error': 'Invalid quantity'}), 400
    except ValueError:
        return jsonify({'error': 'Invalid quantity format'}), 400
@app.route('/place-order', methods=['POST'])
def place_order():
    if 'user_id' not in session:
        return redirect(url_for('login'))

    # Get cart items
    cart_items = Cart.query.filter_by(user_id=session['user_id']).all()
    if not cart_items:
        flash('Your cart is empty!', 'error')
        return redirect(url_for('cart'))

    # Get form data
    name = request.form.get('name')
    email = request.form.get('email')
    phone = request.form.get('phone')
    address = request.form.get('address')

```

```

city = request.form.get('city')
state = request.form.get('state')
pincode = request.form.get('pincode')
payment_method = request.form.get('payment_method')

if not all([name, email, phone, address, city, state, pincode, payment_method]):
    flash('Please fill in all required fields', 'error')
    return redirect(url_for('order_details'))

# Calculate total amount
total_amount = sum(item.price * item.quantity for item in cart_items)

# Create order
order = Order(
    user_id=session['user_id'],
    total_amount=total_amount,
    created_at=datetime.now(),
    shipping_name=name,
    shipping_email=email,
    shipping_phone=phone,
    shipping_address=address,
    shipping_city=city,
    shipping_state=state,
    shipping_pincode=pincode,
    payment_method=payment_method,
    payment_status='pending'
)

try:
    # Add order to database
    db.session.add(order)
    db.session.flush() # This assigns an ID to the order

    # Create order items
    for cart_item in cart_items:
        order_item = OrderItem(
            order_id=order.id,
            product_id=cart_item.product_id,
            quantity=cart_item.quantity,
            size=cart_item.size,
            price=cart_item.price
        )
        db.session.add(order_item)

    # Clear cart
    for item in cart_items:
        db.session.delete(item)

    db.session.commit()

```

```

        # Show appropriate message based on payment method
        if payment_method == 'cod':
            flash('Order placed successfully! You will pay ₹{} on
delivery.'.format(total_amount), 'success')
        elif payment_method == 'upi':
            flash('Order placed successfully! Please complete UPI payment of
₹{}'.format(total_amount), 'success')
        else: # card payment
            flash('Order placed successfully! Please complete card payment of
₹{}'.format(total_amount), 'success')

        return redirect(url_for('my_orders'))

except Exception as e:
    db.session.rollback()
    app.logger.error(f'Error placing order: {str(e)}')
    flash('Error placing order. Please try again.', 'error')
    return redirect(url_for('cart'))

@app.route('/product/<int:product_id>')
def product_detail(product_id):
    if 'user_id' not in session:
        flash('Please login to view product details', 'error')
        return redirect(url_for('login'))
    product = Product.query.get_or_404(product_id)
    return render_template('product_detail.html', product=product)

@app.route('/try_on/<int:product_id>', methods=['GET', 'POST'])
def try_on(product_id):
    if 'user_id' not in session:
        flash('Please login to use virtual try-on', 'error')
        return redirect(url_for('login'))

    product = Product.query.get_or_404(product_id)
    result_image = None
    error = None

    if request.method == 'POST':
        try:
            print("\n=== Starting Virtual Try-On Process ===")

            if 'photo' not in request.files:
                raise ValueError('No photo uploaded')

            photo = request.files['photo']
            if photo.filename == "":
                raise ValueError('No photo selected')

```

```

if photo and allowed_file(photo.filename):
    # Create upload directory if it doesn't exist
    upload_dir = os.path.join(app.root_path, 'static', 'uploads')
    os.makedirs(upload_dir, exist_ok=True)

    # Save uploaded photo with timestamp
    timestamp = int(time.time())
    filename = f'{timestamp}_{secure_filename(photo.filename)}'
    photo_path = os.path.join(upload_dir, filename)
    photo.save(photo_path)
    print(f'✓ Saved user photo to: {photo_path}')

    # Get product image path
    product_path = os.path.join(app.root_path, 'static', product.image_url)
    if not os.path.exists(product_path):
        raise ValueError(f'Product image not found: {product.image_url}')
    print(f'✓ Found product image at: {product_path}')

    # Generate result path
    result_filename = f'result_{timestamp}_{secure_filename(photo.filename)}'
    result_path = os.path.join(upload_dir, result_filename)
    print(f'✓ Will save result to: {result_path}')

    # Process try-on
    tryon_system = EnhancedVirtualTryOn()
    success = tryon_system.try_on(photo_path, product_path, result_path)

    if success and os.path.exists(result_path):
        # Get relative path for URL
        result_rel_path = os.path.relpath(result_path, os.path.join(app.root_path,
'static'))
        result_rel_path = result_rel_path.replace('\\', '/') # Fix Windows paths
        result_image = url_for('static', filename=result_rel_path)
        print(f'✓ Success! Result available at: {result_image}')
        flash('Virtual try-on completed successfully!', 'success')
    else:
        raise ValueError('Failed to generate try-on result')
else:
    raise ValueError('Invalid file type. Please use .png, .jpg, .jpeg, or .gif files.')

except Exception as e:
    error = str(e)
    print(f'✗ Error: {error}')
    flash(f'Error: {error}', 'error')

return render_template('try_on.html',
    product=product,
    result_image=result_image,

```

```

        error=error)

@app.route('/try_on_realtime/<int:product_id>', methods=['POST'])
def try_on_realtime(product_id):
    try:
        if 'frame' not in request.files:
            return jsonify({'error': 'No frame provided'}), 400

        frame_file = request.files['frame']
        if frame_file.filename == "":
            return jsonify({'error': 'No frame selected'}), 400

        # Get the product
        product = Product.query.get_or_404(product_id)

        # Save the frame temporarily
        frame_path = os.path.join(app.config['UPLOAD_FOLDER'], 'temp_frame.jpg')
        frame_file.save(frame_path)

        # Read the frame
        frame = cv2.imread(frame_path)
        if frame is None:
            return jsonify({'error': 'Could not read frame'}), 400

        # Get the clothing image path
        clothing_path = os.path.join(app.root_path, 'static', product.image_url)

        # Process the frame with body detection and clothing overlay
        try:
            processed_frame = body_detector.process_frame(frame, clothing_path)

            # Save the processed frame
            result_filename = f'result_{int(time.time())}.jpg'
            result_path = os.path.join(app.config['UPLOAD_FOLDER'], 'results',
result_filename)
            cv2.imwrite(result_path, processed_frame)

            # Return the processed image path
            return jsonify({
                'result_image': url_for('static', filename=f'uploads/results/{result_filename}')
            })

        except Exception as e:
            return jsonify({'error': f'Processing error: {str(e)}'}), 500

    except Exception as e:
        return jsonify({'error': str(e)}), 500

@app.route('/start_virtual_tryon', methods=['POST'])

```

```

def start_virtual_tryon():
    """Start real-time virtual try-on session"""
    if 'clothing' not in request.files:
        return jsonify({'error': 'No clothing image provided'}), 400

    file = request.files['clothing']
    if file.filename == "" or not allowed_file(file.filename):
        return jsonify({'error': 'Invalid file format'}), 400

    # Save clothing image
    filename = secure_filename(file.filename)
    clothing_path = os.path.join(app.config['UPLOAD_FOLDER'], 'clothing', filename)
    os.makedirs(os.path.dirname(clothing_path), exist_ok=True)
    file.save(clothing_path)

    # Store clothing path in session
    session['current_clothing'] = clothing_path

    # Start pose processor
    pose_processor.start_processing()

    return jsonify({'success': True, 'message': 'Virtual try-on session started'})

@app.route('/stop_virtual_tryon', methods=['POST'])
def stop_virtual_tryon():
    """Stop virtual try-on session"""
    pose_processor.stop_processing()
    session.pop('current_clothing', None)
    return jsonify({'success': True, 'message': 'Virtual try-on session stopped'})

@app.route('/process_frame', methods=['POST'])
def process_frame():
    """Process a single frame for virtual try-on with 3D pose estimation"""
    try:
        # Get frame data
        frame_data = request.get_json()
        frame = np.array(frame_data['frame'])

        # Get current clothing image
        clothing_path = session.get('current_clothing')
        if not clothing_path:
            return jsonify({'error': 'No clothing selected'}), 400

        clothing_img = cv2.imread(clothing_path, cv2.IMREAD_UNCHANGED)
        if clothing_img is None:
            return jsonify({'error': 'Failed to load clothing image'}), 500

        # Process frame with 3D pose estimation
        processed_frame, fit_metrics, heatmap = clothing_adjuster.adjust_clothing(

```

```

        frame,
        clothing_img,
        None # landmarks will be handled inside adjust_clothing now
    )

    if processed_frame is None:
        return jsonify({'error': 'Failed to detect pose'}), 400

    # Encode processed frame and heatmap
    _, frame_buffer = cv2.imencode('.jpg', processed_frame)
    frame_data = base64.b64encode(frame_buffer).decode('utf-8')

    _, heatmap_buffer = cv2.imencode('.jpg', heatmap)
    heatmap_data = base64.b64encode(heatmap_buffer).decode('utf-8')

    # Prepare response with 3D transformation data
    response_data = {
        'processed_frame': frame_data,
        'heatmap': heatmap_data,
        'fit_analysis': {
            'overall_score': fit_metrics.overall_score if fit_metrics else 0,
            'shoulder_alignment': fit_metrics.shoulder_alignment if fit_metrics else 0,
            'chest_fit': fit_metrics.chest_fit if fit_metrics else 0,
            'waist_fit': fit_metrics.waist_fit if fit_metrics else 0,
            'arm_holes': fit_metrics.arm_holes if fit_metrics else 0,
            'problem_areas': fit_metrics.problem_areas if fit_metrics else []
        }
    }

    return jsonify(response_data)

except Exception as e:
    return jsonify({'error': str(e)}), 500

@app.route('/download_result/<path:filename>')
def download_result(filename):
    try:
        # Construct the full path to the result image
        upload_dir = os.path.join(app.root_path, 'static', 'uploads')
        return send_from_directory(
            upload_dir,
            filename,
            as_attachment=True,
            download_name='virtual-tryon-result.jpg'
        )
    except Exception as e:
        flash('Error downloading the result: ' + str(e), 'error')
        return redirect(url_for('index'))

```

```

@app.route('/process_3d_model', methods=['POST'])
def process_3d_model():
    if 'model' not in request.files:
        return jsonify({'error': 'No model file provided'}), 400

    file = request.files['model']
    if file.filename == "" or not allowed_file(file.filename):
        return jsonify({'error': 'Invalid file format'}), 400

    # Save the uploaded model
    filename = secure_filename(file.filename)
    model_path = os.path.join(app.config['UPLOAD_FOLDER'], 'models', filename)
    os.makedirs(os.path.dirname(model_path), exist_ok=True)
    file.save(model_path)

    try:
        # Process the 3D model
        processor = ClothingModelProcessor(model_path)
        processor.load_model()

        # Handle body mesh if provided
        if 'body_mesh' in request.files:
            body_mesh_file = request.files['body_mesh']
            body_mesh_filename = secure_filename(body_mesh_file.filename)
            body_mesh_path = os.path.join(app.config['UPLOAD_FOLDER'], 'models',
            'body_' + body_mesh_filename)
            body_mesh_file.save(body_mesh_path)
            processor.load_body_mesh(body_mesh_path)

        # Handle texture if provided
        if 'texture' in request.files:
            texture_file = request.files['texture']
            texture_filename = secure_filename(texture_file.filename)
            texture_path = os.path.join(app.config['UPLOAD_FOLDER'], 'textures',
            texture_filename)
            os.makedirs(os.path.dirname(texture_path), exist_ok=True)
            texture_file.save(texture_path)
            processor.apply_texture_mapping(texture_path)

        # Apply processing steps based on request parameters
        if 'scale' in request.form:
            scale_factor = float(request.form['scale'])
            processor.adjust_scale(scale_factor)

        if 'smooth' in request.form:
            iterations = int(request.form['smooth'])
            lambda_factor = float(request.form.get('lambda_factor', 0.5))
            processor.smooth_surface(iterations, lambda_factor)

```

```

# Handle mesh deformation if constraint points are provided
if 'constraint_points' in request.form and 'target_points' in request.form:
    constraint_points = np.array(json.loads(request.form['constraint_points']))
    target_points = np.array(json.loads(request.form['target_points']))
    weight = float(request.form.get('deform_weight', 1.0))
    processor.deform_to_body(constraint_points, target_points, weight)

# Export processed model
output_filename = f'processed_{filename}'
output_path = os.path.join(app.config['UPLOAD_FOLDER'], 'models',
output_filename)
processor.export_model(output_path)

return jsonify({
    'success': True,
    'message': 'Model processed successfully',
    'processed_model': output_filename
})

except Exception as e:
    return jsonify({'error': str(e)}), 500

@app.route('/generate_surface', methods=['POST'])
def generate_surface():
    """Generate a 3D surface visualization"""
    try:
        # Create output directory if it doesn't exist
        output_dir = os.path.join(app.config['UPLOAD_FOLDER'], 'visualizations')
        os.makedirs(output_dir, exist_ok=True)

        # Get visualization type from request
        use_plotly = request.form.get('use_plotly', 'false').lower() == 'true'

        # Generate visualization
        output_path = create_visualization(output_dir, use_plotly)

        # Get the filename from the path
        filename = os.path.basename(output_path)

        return jsonify({
            'success': True,
            'message': 'Surface visualization generated successfully',
            'file_url': url_for('static', filename=f'uploads/visualizations/{filename}')
        })

    except Exception as e:
        return jsonify({'error': str(e)}), 500

@app.after_request

```

```

def add_header(response):
    """Ensure proper caching for dynamic images"""
    if 'image' in response.mimetype:
        response.headers['Cache-Control'] = 'no-store, no-cache, must-revalidate, max-age=0'
    return response

@app.route('/order-details')
def order_details():
    if 'user_id' not in session:
        return redirect(url_for('login'))

    # Get cart items with product information
    cart_items = Cart.query.filter_by(user_id=session['user_id']).all()
    if not cart_items:
        flash('Your cart is empty', 'error')
        return redirect(url_for('cart'))

    # Prepare cart items with product details
    items_with_details = []
    total_price = 0

    for item in cart_items:
        product = Product.query.get(item.product_id)
        if product:
            item_details = {
                'name': product.name,
                'selected_size': item.size,
                'quantity': item.quantity,
                'price': item.price * item.quantity,
                'price_display': price_display(item.price * item.quantity)
            }
            items_with_details.append(item_details)
            total_price += item.price * item.quantity

    # Get user info for pre-filling
    user = User.query.get(session['user_id'])

    return render_template('order_details.html',
                           cart_items=items_with_details,
                           total_price=total_price,
                           total_price_display=price_display(total_price),
                           user=user)

@app.route('/my-orders')
def my_orders():
    if 'user_id' not in session:
        flash('Please login to view your orders', 'error')
        return redirect(url_for('login'))

```

```

# Get all orders for the current user
orders =
Order.query.filter_by(user_id=session['user_id']).order_by(Order.created_at.desc()).all()
return render_template('my_orders.html', orders=orders)

if __name__ == '__main__':
    init_db()
    app.run(debug=True)

```

- **body_detection.py**

```

import cv2
import mediapipe as mp
import numpy as np

class BodyDetector:
    def __init__(self):
        self.mp_pose = mp.solutions.pose
        self.pose = self.mp_pose.Pose(
            static_image_mode=False,
            model_complexity=2, # Increased model complexity for better accuracy
            min_detection_confidence=0.5,
            min_tracking_confidence=0.5
        )

    def detect_body(self, image):
        # Convert to RGB for MediaPipe
        image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        results = self.pose.process(image_rgb)
        return results

    def overlay_clothing(self, image, clothing_image, pose_landmarks):
        if not pose_landmarks:
            return image

        ih, iw, _ = image.shape

        try:
            # Get key body landmarks
            l_shoulder =
(int(pose_landmarks.landmark[self.mp_pose.PoseLandmark.LEFT_SHOULDER].x * iw),
int(pose_landmarks.landmark[self.mp_pose.PoseLandmark.LEFT_SHOULDER].y * ih))
            r_shoulder =
(int(pose_landmarks.landmark[self.mp_pose.PoseLandmark.RIGHT_SHOULDER].x *
iw),

```

```

int(pose_landmarks.landmark[self.mp_pose.PoseLandmark.RIGHT_SHOULDER].y * ih))
    l_hip = (int(pose_landmarks.landmark[self.mp_pose.PoseLandmark.LEFT_HIP].x
* iw),
            int(pose_landmarks.landmark[self.mp_pose.PoseLandmark.LEFT_HIP].y *
ih))
    r_hip =
(int(pose_landmarks.landmark[self.mp_pose.PoseLandmark.RIGHT_HIP].x * iw),
    int(pose_landmarks.landmark[self.mp_pose.PoseLandmark.RIGHT_HIP].y *
ih))

# Calculate clothing dimensions based on body
shoulder_width = int(np.sqrt((r_shoulder[0] - l_shoulder[0])**2 +
                             (r_shoulder[1] - l_shoulder[1])**2))
torso_height = int(np.sqrt((l_shoulder[0] - l_hip[0])**2 +
                             (l_shoulder[1] - l_hip[1])**2))

# Add padding to dimensions
shoulder_width = int(shoulder_width * 1.2) # 20% wider
torso_height = int(torso_height * 1.1) # 10% taller

# Read and preprocess clothing image
if isinstance(clothing_image, str):
    clothing_image = cv2.imread(clothing_image, cv2.IMREAD_UNCHANGED)
    if clothing_image is None:
        raise ValueError("Could not load clothing image")

# Create alpha channel if not present
if clothing_image.shape[2] == 3:
    clothing_image = cv2.cvtColor(clothing_image, cv2.COLOR_BGR2BGRA)

# Resize clothing to match body dimensions
clothing_resized = cv2.resize(clothing_image, (shoulder_width, torso_height))

# Calculate position to place clothing
center_x = (l_shoulder[0] + r_shoulder[0]) // 2
center_y = (l_shoulder[1] + r_shoulder[1]) // 2

# Calculate top-left corner for overlay
x_offset = max(0, center_x - clothing_resized.shape[1] // 2)
y_offset = max(0, center_y - clothing_resized.shape[0] // 4) # Adjusted to align
better with shoulders

# Ensure we don't go out of bounds
if x_offset + clothing_resized.shape[1] > iw:
    clothing_resized = clothing_resized[:, :iw - x_offset]
if y_offset + clothing_resized.shape[0] > ih:
    clothing_resized = clothing_resized[:ih - y_offset, :]

```

```

        # Get the alpha channel
        alpha = clothing_resized[:, :, 3] / 255.0
        alpha = np.stack([alpha] * 3, axis=-1)

        # Get the region of interest (ROI)
        roi = image[y_offset:y_offset + clothing_resized.shape[0],
                    x_offset:x_offset + clothing_resized.shape[1]]

        # Blend the clothing with the image using alpha channel
        blended = (1 - alpha) * roi + alpha * clothing_resized[:, :, :3]

        # Put the blended region back into the image
        image[y_offset:y_offset + clothing_resized.shape[0],
              x_offset:x_offset + clothing_resized.shape[1]] = blended

    except Exception as e:
        print(f"Error overlaying clothing: {str(e)}")
        return image

    return image

def process_frame(self, frame, clothing_path):
    # Detect body in frame
    pose_results = self.detect_body(frame)

    if pose_results.pose_landmarks:
        # Overlay clothing on the detected body
        frame = self.overlay_clothing(frame.copy(), clothing_path,
        pose_results.pose_landmarks)

    return frame

```

- **tryon_utils.py**

```

import cv2
import numpy as np
import mediapipe as mp
import os
import sys

# Windows console encoding fix
if sys.platform.startswith('win'):
    import locale
    if sys.stdout.encoding != 'utf-8':
        sys.stdout.reconfigure(encoding='utf-8')

```

```

class EnhancedVirtualTryOn:
    def __init__(self):
        self.mp_pose = mp.solutions.pose
        self.pose = self.mp_pose.Pose(
            static_image_mode=True,
            model_complexity=2,
            min_detection_confidence=0.5
        )

    def preprocess_image(self, image):
        """Preprocess image for better visibility"""
        if image.shape[2] == 3: # RGB/BGR image
            # Convert to RGB
            image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

            # Apply CLAHE to enhance contrast
            lab = cv2.cvtColor(image, cv2.COLOR_RGB2LAB)
            l, a, b = cv2.split(lab)
            clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8,8))
            l = clahe.apply(l)
            lab = cv2.merge((l,a,b))
            image = cv2.cvtColor(lab, cv2.COLOR_LAB2RGB)

            # Enhance saturation and brightness
            hsv = cv2.cvtColor(image, cv2.COLOR_RGB2HSV)
            h, s, v = cv2.split(hsv)
            s = cv2.multiply(s, 1.4).astype(np.uint8) # Increase saturation
            v = cv2.multiply(v, 1.2).astype(np.uint8) # Increase brightness
            hsv = cv2.merge([h, s, v])
            image = cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB)

        return image

    def preprocess_clothing(self, clothing_img):
        """Preprocess clothing image to remove background and enhance visibility"""
        try:
            # Convert to RGBA if needed
            if clothing_img.shape[2] == 3:
                clothing_img = cv2.cvtColor(clothing_img, cv2.COLOR_BGR2BGRA)

            # Create binary mask for non-white pixels
            gray = cv2.cvtColor(clothing_img, cv2.COLOR_BGR2GRAY)
            _, binary = cv2.threshold(gray, 250, 255, cv2.THRESH_BINARY_INV)

            # Remove noise
            kernel = np.ones((3,3), np.uint8)
            binary = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)

            # Create alpha channel from binary mask

```

```

clothing_img[:, :, 3] = binary

# Enhance clothing visibility
clothing_rgb = clothing_img[:, :, :3].copy()
clothing_lab = cv2.cvtColor(clothing_rgb, cv2.COLOR_BGR2LAB)
l, a, b = cv2.split(clothing_lab)

# Apply CLAHE to L channel
clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8,8))
l = clahe.apply(l)

# Merge back
clothing_lab = cv2.merge((l,a,b))
clothing_rgb = cv2.cvtColor(clothing_lab, cv2.COLOR_LAB2BGR)

# Update RGB channels while preserving alpha
clothing_img[:, :, :3] = clothing_rgb

return clothing_img

except Exception as e:
    print(f"[ERROR] In preprocess_clothing: {str(e)}")
    return None

def detect_body_features(self, image):
    """Detect body landmarks and create body mask"""
    try:
        # Convert to RGB for MediaPipe
        image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        # Get pose landmarks
        with self.mp_pose.Pose(
            static_image_mode=True,
            model_complexity=2,
            min_detection_confidence=0.5,
            enable_segmentation=True # Enable segmentation for better masking
        ) as pose:
            results = pose.process(image_rgb)

        if not results.pose_landmarks:
            print("[ERROR] No pose landmarks detected")
            return None, None

        # Get relevant landmark points for upper body
        height, width = image.shape[:2]
        points = []
        upper_body_landmarks = [
            self.mp_pose.PoseLandmark.LEFT_SHOULDER,
            self.mp_pose.PoseLandmark.RIGHT_SHOULDER,

```

```

        self.mp_pose.PoseLandmark.LEFT_ELBOW,
        self.mp_pose.PoseLandmark.RIGHT_ELBOW,
        self.mp_pose.PoseLandmark.LEFT_HIP,
        self.mp_pose.PoseLandmark.RIGHT_HIP,
        self.mp_pose.PoseLandmark.LEFT_WRIST,
        self.mp_pose.PoseLandmark.RIGHT_WRIST
    ]

    for landmark in upper_body_landmarks:
        x = int(results.pose_landmarks.landmark[landmark.value].x * width)
        y = int(results.pose_landmarks.landmark[landmark.value].y * height)
        points.append([x, y])

    points = np.array(points, np.int32)

    # Create refined mask for upper body region
    mask = np.zeros(image.shape[:2], dtype=np.uint8)

    # Create a smoother hull by adding interpolated points
    hull = cv2.convexHull(points)
    smooth_hull = []
    for i in range(len(hull)):
        pt1 = hull[i][0]
        pt2 = hull[(i + 1) % len(hull)][0]
        # Add intermediate points
        for t in range(0, 100, 10):
            x = int(pt1[0] + (pt2[0] - pt1[0]) * t / 100)
            y = int(pt1[1] + (pt2[1] - pt1[1]) * t / 100)
            smooth_hull.append([x, y])

    smooth_hull = np.array(smooth_hull, dtype=np.int32)
    cv2.fillConvexPoly(mask, smooth_hull, 255)

    # Apply Gaussian blur to smooth the mask edges
    mask = cv2.GaussianBlur(mask, (5, 5), 0)

    # Dilate mask to ensure coverage
    kernel = np.ones((15,15), np.uint8) # Reduced kernel size
    mask = cv2.dilate(mask, kernel, iterations=1)

    return results.pose_landmarks.landmark, mask

except Exception as e:
    print(f"[ERROR] In detect_body_features: {str(e)}")
    return None, None

def get_body_measurements(self, landmarks, image_shape):
    """Get body measurements from landmarks"""
    try:

```

```

height, width = image_shape[:2]

# Get key points without visualization
left_shoulder =
(int(landmarks[self.mp_pose.PoseLandmark.LEFT_SHOULDER.value].x * width),
 int(landmarks[self.mp_pose.PoseLandmark.LEFT_SHOULDER.value].y
 * height))
right_shoulder =
(int(landmarks[self.mp_pose.PoseLandmark.RIGHT_SHOULDER.value].x * width),
 int(landmarks[self.mp_pose.PoseLandmark.RIGHT_SHOULDER.value].y * height))
left_hip = (int(landmarks[self.mp_pose.PoseLandmark.LEFT_HIP.value].x *
width),
            int(landmarks[self.mp_pose.PoseLandmark.LEFT_HIP.value].y * height))
right_hip = (int(landmarks[self.mp_pose.PoseLandmark.RIGHT_HIP.value].x *
width),
             int(landmarks[self.mp_pose.PoseLandmark.RIGHT_HIP.value].y * height))

# Calculate measurements silently
shoulder_width = np.sqrt((right_shoulder[0] - left_shoulder[0])**2 +
                        (right_shoulder[1] - left_shoulder[1])**2)
torso_height = np.sqrt((left_hip[0] - left_shoulder[0])**2 +
                      (left_hip[1] - left_shoulder[1])**2)

# Calculate center points
shoulder_center = ((left_shoulder[0] + right_shoulder[0])//2,
                  (left_shoulder[1] + right_shoulder[1])//2)
hip_center = ((left_hip[0] + right_hip[0])//2,
              (left_hip[1] + right_hip[1])//2)

# Calculate body angle
body_angle = np.arctan2(hip_center[1] - shoulder_center[1],
                        hip_center[0] - shoulder_center[0])

return {
    'shoulder_width': shoulder_width,
    'torso_height': torso_height,
    'shoulder_center': shoulder_center,
    'hip_center': hip_center,
    'body_angle': body_angle,
    'left_shoulder': left_shoulder,
    'right_shoulder': right_shoulder,
    'left_hip': left_hip,
    'right_hip': right_hip
}

except Exception as e:
    print(f'[ERROR] In get_body_measurements: {str(e)}')
    return None

```

```

def warp_clothing_to_body(self, clothing_img, measurements, target_shape):
    """Warp clothing image to match body measurements"""
    try:
        if measurements is None:
            return None

        height, width = target_shape[:2]

        # Calculate target width based on shoulder width with increased size
        target_width = int(measurements['shoulder_width'] * 1.8) # Increased from 1.5

        # Calculate target height while maintaining aspect ratio
        aspect_ratio = clothing_img.shape[0] / clothing_img.shape[1]
        target_height = int(target_width * aspect_ratio * 1.2) # Increased from 0.9

        # Ensure height doesn't exceed torso with more allowance
        max_height = int(measurements['torso_height'] * 1.4) # Increased from 1.05
        if target_height > max_height:
            target_height = max_height
            target_width = int(target_height / (aspect_ratio * 1.2))

        # Create perspective transform points
        src_points = np.float32([
            [0, 0],
            [clothing_img.shape[1], 0],
            [0, clothing_img.shape[0]],
            [clothing_img.shape[1], clothing_img.shape[0]]
        ])

        # Calculate shoulder slope with dampening
        shoulder_slope = (measurements['right_shoulder'][1] -
measurements['left_shoulder'][1]) /\
            (measurements['right_shoulder'][0] - measurements['left_shoulder'][0] +
1e-6)
        shoulder_slope *= 0.5 # Further reduce slope effect

        # Adjust target points based on body measurements
        shoulder_center = measurements['shoulder_center']
        hip_center = measurements['hip_center']

        # Calculate vertical positioning with higher placement
        vertical_offset = int(target_height * 0.15) # Increased from 0.05

        # Calculate horizontal positioning with reduced angle compensation
        body_angle = measurements['body_angle']
        angle_compensation = np.tan(body_angle) * target_height * 0.1 # Reduced from
0.2

```

```

# Calculate base points with wider spread
top_left = [
    shoulder_center[0] - target_width//2 + int(angle_compensation),
    shoulder_center[1] - vertical_offset
]
top_right = [
    shoulder_center[0] + target_width//2 + int(angle_compensation),
    shoulder_center[1] - vertical_offset
]
bottom_left = [
    hip_center[0] - target_width//2 - int(angle_compensation),
    hip_center[1] + vertical_offset # Added positive offset for length
]
bottom_right = [
    hip_center[0] + target_width//2 - int(angle_compensation),
    hip_center[1] + vertical_offset # Added positive offset for length
]

# Apply shoulder slope with reduced effect
slope_offset = int(target_width * shoulder_slope * 0.1) # Reduced from 0.2
top_left[1] -= slope_offset
top_right[1] += slope_offset

# Add slight curve to bottom points for better fit
bottom_curve = int(target_width * 0.03) # Reduced from 0.05
bottom_left[0] += bottom_curve
bottom_right[0] -= bottom_curve

dst_points = np.float32([top_left, top_right, bottom_left, bottom_right])

# Get perspective transform
matrix = cv2.getPerspectiveTransform(src_points, dst_points)

# Apply transform with improved interpolation
warped = cv2.warpPerspective(
    clothing_img,
    matrix,
    (width, height),
    flags=cv2.INTER_CUBIC, # Changed to CUBIC for better quality
    borderMode=cv2.BORDER_REPLICATE
)

return warped

except Exception as e:
    print(f"[ERROR] In warp_clothing_to_body: {str(e)}")
    return None

def get_clothing_position(self, landmarks, clothing_shape, image_shape):

```

```

"""Calculate optimal position for clothing overlay"""
try:
    height, width = image_shape[:2]
    clothing_height, clothing_width = clothing_shape[:2]

    # Get shoulder points
    left_shoulder_x =
int(landmarks[self.mp_pose.PoseLandmark.LEFT_SHOULDER.value].x * width)
    left_shoulder_y =
int(landmarks[self.mp_pose.PoseLandmark.LEFT_SHOULDER.value].y * height)
    right_shoulder_x =
int(landmarks[self.mp_pose.PoseLandmark.RIGHT_SHOULDER.value].x * width)
    right_shoulder_y =
int(landmarks[self.mp_pose.PoseLandmark.RIGHT_SHOULDER.value].y * height)

    # Calculate center position
    center_x = (left_shoulder_x + right_shoulder_x) // 2
    center_y = (left_shoulder_y + right_shoulder_y) // 2

    # Calculate position to place clothing
    x = center_x - (clothing_width // 2)
    y = center_y - int(clothing_height * 0.35) # Move up to cover shoulders better

    # Ensure within image bounds
    x = max(0, min(x, width - clothing_width))
    y = max(0, min(y, height - clothing_height))

    return (x, y)

except Exception as e:
    print(f"[ERROR] In get_clothing_position: {str(e)}")
    return (0, 0)

def blend_images(self, clothing, person, position, body_mask):
    """Blend clothing onto person image with improved alpha blending"""
    try:
        if clothing is None or person is None:
            return person

        # Create output image
        result = person.copy()

        # Get clothing alpha channel with enhanced opacity
        alpha = clothing[:, :, 3] / 255.0 if clothing.shape[2] == 4 else
np.ones(clothing.shape[:2])
        alpha = np.clip(alpha * 1.3, 0, 1) # Increase opacity by 30%

        # Apply Gaussian blur for smoother edges
        alpha = cv2.GaussianBlur(alpha, (3, 3), 0)

```

```

# Get clothing RGB channels
clothing_rgb = clothing[:, :, :3]

# Enhance clothing visibility
clothing_lab = cv2.cvtColor(clothing_rgb, cv2.COLOR_BGR2LAB)
l, a, b = cv2.split(clothing_lab)

# Apply CLAHE to L channel
clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8,8))
l = clahe.apply(l)

# Merge back
clothing_lab = cv2.merge((l,a,b))
clothing_rgb = cv2.cvtColor(clothing_lab, cv2.COLOR_LAB2BGR)

# Enhance contrast
clothing_rgb = cv2.convertScaleAbs(clothing_rgb, alpha=1.1, beta=5)

# Apply color correction to match lighting
person_hsv = cv2.cvtColor(person, cv2.COLOR_BGR2HSV)
clothing_hsv = cv2.cvtColor(clothing_rgb, cv2.COLOR_BGR2HSV)

# Match brightness while maintaining contrast
avg_v_person = np.mean(person_hsv[:, :, 2])
avg_v_clothing = np.mean(clothing_hsv[:, :, 2])
brightness_ratio = min(avg_v_person / (avg_v_clothing + 1e-6), 1.2) # Cap at
120%

clothing_hsv[:, :, 2] = np.clip(clothing_hsv[:, :, 2] * brightness_ratio, 0, 255)
clothing_rgb = cv2.cvtColor(clothing_hsv, cv2.COLOR_HSV2BGR)

# Create alpha channels for blending
alpha_3d = np.stack([alpha] * 3, axis=2)

# Apply body mask to clothing mask
body_mask_roi = body_mask[position[1]:position[1]+clothing.shape[0],
position[0]:position[0]+clothing.shape[1]] / 255.0
alpha_3d = alpha_3d * body_mask_roi

# Blend images with enhanced visibility
blended = cv2.addWeighted(
    (clothing_rgb * alpha_3d).astype(np.uint8),
    1.0, # Increased weight for clothing
    (result[position[1]:position[1]+clothing.shape[0],
position[0]:position[0]+clothing.shape[1]] * (1 - alpha_3d)).astype(np.uint8),
    0.8, # Reduced weight for original image
    0
)

```

```

        # Copy only the valid region with feathered edges
        valid_region = (alpha_3d > 0.1).any(axis=2) # Reduced threshold
        result[position[1]:position[1]+clothing.shape[0],
position[0]:position[0]+clothing.shape[1]][valid_region] = blended[valid_region]

    return result

except Exception as e:
    print(f"[ERROR] In blend_images: {str(e)}")
    return person

def try_on(self, person_image_path, clothing_image_path, output_path):
    """Main virtual try-on function with improved body fitting"""
    try:
        # Load images
        person_img = cv2.imread(person_image_path)
        if person_img is None:
            raise ValueError("Could not load person image")

        clothing_img = cv2.imread(clothing_image_path, cv2.IMREAD_UNCHANGED)
        if clothing_img is None:
            raise ValueError("Could not load clothing image")

        # Convert clothing to RGBA if needed
        if clothing_img.shape[2] == 3:
            clothing_img = cv2.cvtColor(clothing_img, cv2.COLOR_BGR2BGRA)

        # Keep original person image
        original_person = person_img.copy()

        # Get body features and measurements
        landmarks, body_mask = self.detect_body_features(person_img)
        if landmarks is None:
            raise ValueError("No body landmarks detected - please ensure full upper body is
visible")

        # Get image dimensions
        height, width = person_img.shape[:2]

        # Get key body points with proper scaling
        left_shoulder = (
            int(landmarks[self.mp_pose.PoseLandmark.LEFT_SHOULDER.value].x *
width),
            int(landmarks[self.mp_pose.PoseLandmark.LEFT_SHOULDER.value].y *
height)
        )
        right_shoulder = (
            int(landmarks[self.mp_pose.PoseLandmark.RIGHT_SHOULDER.value].x *

```

```

width),
    int(landmarks[self.mp_pose.PoseLandmark.RIGHT_SHOULDER.value].y *
height)
)
left_hip = (
    int(landmarks[self.mp_pose.PoseLandmark.LEFT_HIP.value].x * width),
    int(landmarks[self.mp_pose.PoseLandmark.LEFT_HIP.value].y * height)
)
right_hip = (
    int(landmarks[self.mp_pose.PoseLandmark.RIGHT_HIP.value].x * width),
    int(landmarks[self.mp_pose.PoseLandmark.RIGHT_HIP.value].y * height)
)

# Calculate measurements
shoulder_width = int(np.sqrt((right_shoulder[0] - left_shoulder[0])**2 +
                             (right_shoulder[1] - left_shoulder[1])**2))

shoulder_center = (
    (left_shoulder[0] + right_shoulder[0])/2,
    (left_shoulder[1] + right_shoulder[1])/2
)

hip_center = (
    (left_hip[0] + right_hip[0])/2,
    (left_hip[1] + right_hip[1])/2
)

torso_height = hip_center[1] - shoulder_center[1]

# Calculate t-shirt dimensions
target_width = int(shoulder_width * 1.8)

# Use original aspect ratio
original_aspect = clothing_img.shape[0] / clothing_img.shape[1]
target_height = int(torso_height * 1.4)
target_width = int(target_height / original_aspect)

# Resize t-shirt
resized_clothing = cv2.resize(clothing_img, (target_width, target_height),
                              interpolation=cv2.INTER_LANCZOS4)

# Position t-shirt
x_offset = shoulder_center[0] - target_width//2
y_offset = shoulder_center[1] - int(target_height * 0.1)

# Apply minimal rotation
body_angle = np.arctan2(right_shoulder[1] - left_shoulder[1],
                        right_shoulder[0] - left_shoulder[0])
x_offset += int(np.tan(body_angle) * target_height * 0.03)

```

```

# Ensure within bounds
x_offset = max(0, min(x_offset, person_img.shape[1] - target_width))
y_offset = max(0, min(y_offset, person_img.shape[0] - target_height))

# Create mask
mask = resized_clothing[:, :, 3] if resized_clothing.shape[2] == 4 else
np.ones(resized_clothing.shape[:2])
mask = mask / 255.0
mask = cv2.GaussianBlur(mask, (3, 3), 0)

# Create output image
result = original_person.copy()

# Calculate valid region for placement
y1, y2 = y_offset, y_offset + resized_clothing.shape[0]
x1, x2 = x_offset, x_offset + resized_clothing.shape[1]

# Ensure coordinates are within image bounds
y1 = max(0, y1)
y2 = min(result.shape[0], y2)
x1 = max(0, x1)
x2 = min(result.shape[1], x2)

# Adjust clothing region to match valid coordinates
clothing_y1 = 0 if y1 == y_offset else y_offset - y1
clothing_y2 = resized_clothing.shape[0] if y2 == y_offset +
resized_clothing.shape[0] else resized_clothing.shape[0] - (y_offset +
resized_clothing.shape[0] - y2)
clothing_x1 = 0 if x1 == x_offset else x_offset - x1
clothing_x2 = resized_clothing.shape[1] if x2 == x_offset +
resized_clothing.shape[1] else resized_clothing.shape[1] - (x_offset +
resized_clothing.shape[1] - x2)

# Get the region of interest
roi = result[y1:y2, x1:x2]

# Get the clothing region
clothing_roi = resized_clothing[clothing_y1:clothing_y2, clothing_x1:clothing_x2]
mask_roi = mask[clothing_y1:clothing_y2, clothing_x1:clothing_x2]

# Apply body mask to clothing mask
body_mask_roi = body_mask[y1:y2, x1:x2] / 255.0
mask_roi = mask_roi * body_mask_roi

# Blend the images
for c in range(3): # For each color channel
    roi[:, :, c] = roi[:, :, c] * (1 - mask_roi) + clothing_roi[:, :, c] * mask_roi

```

```

# Place the blended region back into the result
result[y1:y2, x1:x2] = roi

# Save result
os.makedirs(os.path.dirname(output_path), exist_ok=True)
if not cv2.imwrite(output_path, result):
    raise ValueError("Failed to save result image")

return True

except Exception as e:
    print(f"[ERROR] In try_on: {str(e)}")
    return False

```

- **fit_analyzer.py**

```

import cv2
import numpy as np
from dataclasses import dataclass
from typing import Dict, List, Tuple, Optional

@dataclass
class FitMetrics:
    shoulder_alignment: float # 0-1 score for shoulder alignment
    chest_fit: float # 0-1 score for chest area fit
    waist_fit: float # 0-1 score for waist area fit
    arm_holes: float # 0-1 score for arm hole positioning
    overall_score: float # Weighted average of all metrics
    problem_areas: List[str] # List of areas needing adjustment

class FitAnalyzer:
    def __init__(self):
        # MediaPipe pose landmark indices
        self.SHOULDER_LEFT = 11
        self.SHOULDER_RIGHT = 12
        self.WAIST_LEFT = 23
        self.WAIST_RIGHT = 24
        self.ELBOW_LEFT = 13
        self.ELBOW_RIGHT = 14
        self.HIP_LEFT = 23
        self.HIP_RIGHT = 24
        self.NECK = 0

        # Fit threshold values
        self.GOOD_FIT_THRESHOLD = 0.85
        self.MODERATE_FIT_THRESHOLD = 0.7
        self.POOR_FIT_THRESHOLD = 0.5

```

```

# Color maps for visualization
self.colors = {
    'good': (0, 255, 0),    # Green
    'moderate': (0, 255, 255), # Yellow
    'poor': (0, 0, 255)     # Red
}

def analyze_fit(self, frame: np.ndarray, clothing_mask: np.ndarray,
                landmarks: List, clothing_points: np.ndarray) -> FitMetrics:
    """Analyze the fit of the clothing based on body landmarks"""
    h, w = frame.shape[:2]

    # Convert landmarks to pixel coordinates
    keypoints = self._get_keypoints(landmarks, w, h)

    # Calculate fit metrics
    metrics = self._calculate_fit_metrics(keypoints, clothing_mask,
    clothing_points)

    return metrics

def _get_keypoints(self, landmarks, width: int, height: int) -> Dict[str,
np.ndarray]:
    """Extract keypoints from landmarks in pixel coordinates"""
    keypoints = {}

    # Extract key body points
    keypoints['left_shoulder'] = np.array([
        landmarks[self.SHOULDER_LEFT].x * width,
        landmarks[self.SHOULDER_LEFT].y * height
    ])
    keypoints['right_shoulder'] = np.array([
        landmarks[self.SHOULDER_RIGHT].x * width,
        landmarks[self.SHOULDER_RIGHT].y * height
    ])
    keypoints['left_waist'] = np.array([
        landmarks[self.WAIST_LEFT].x * width,
        landmarks[self.WAIST_LEFT].y * height
    ])
    keypoints['right_waist'] = np.array([
        landmarks[self.WAIST_RIGHT].x * width,
        landmarks[self.WAIST_RIGHT].y * height
    ])
    keypoints['neck'] = np.array([
        landmarks[self.NECK].x * width,
        landmarks[self.NECK].y * height
    ])

```

```

    return keypoints

def _calculate_fit_metrics(self, keypoints: Dict[str, np.ndarray],
                           clothing_mask: np.ndarray,
                           clothing_points: np.ndarray) -> FitMetrics:
    """Calculate various fit metrics"""
    # Calculate shoulder alignment
    shoulder_alignment = self._calculate_shoulders_alignment(
        keypoints['left_shoulder'],
        keypoints['right_shoulder'],
        clothing_points
    )

    # Calculate chest fit
    chest_fit = self._calculate_chest_fit(
        keypoints,
        clothing_mask
    )

    # Calculate waist fit
    waist_fit = self._calculate_waist_fit(
        keypoints['left_waist'],
        keypoints['right_waist'],
        clothing_mask
    )

    # Calculate arm hole positioning
    arm_holes = self._calculate_arm_holes_fit(
        keypoints,
        clothing_mask
    )

    # Calculate overall score
    weights = {
        'shoulder': 0.3,
        'chest': 0.3,
        'waist': 0.2,
        'arms': 0.2
    }

    overall_score = (
        weights['shoulder'] * shoulder_alignment +
        weights['chest'] * chest_fit +
        weights['waist'] * waist_fit +
        weights['arms'] * arm_holes
    )

    # Identify problem areas
    problem_areas = []

```

```

        if shoulder_alignment < self.MODERATE_FIT_THRESHOLD:
            problem_areas.append('shoulders')
        if chest_fit < self.MODERATE_FIT_THRESHOLD:
            problem_areas.append('chest')
        if waist_fit < self.MODERATE_FIT_THRESHOLD:
            problem_areas.append('waist')
        if arm_holes < self.MODERATE_FIT_THRESHOLD:
            problem_areas.append('arm_holes')

    return FitMetrics(
        shoulder_alignment=shoulder_alignment,
        chest_fit=chest_fit,
        waist_fit=waist_fit,
        arm_holes=arm_holes,
        overall_score=overall_score,
        problem_areas=problem_areas
    )

def generate_heatmap(self, frame: np.ndarray, metrics: FitMetrics,
                    keypoints: Dict[str, np.ndarray]) -> np.ndarray:
    """Generate a heatmap overlay based on fit metrics"""
    h, w = frame.shape[:2]
    heatmap = np.zeros((h, w), dtype=np.float32)

    # Generate heatmap for each body region
    self._add_shoulder_heatmap(heatmap, keypoints, metrics.shoulder_alignment)
    self._add_chest_heatmap(heatmap, keypoints, metrics.chest_fit)
    self._add_waist_heatmap(heatmap, keypoints, metrics.waist_fit)
    self._add_arm_holes_heatmap(heatmap, keypoints, metrics.arm_holes)

    # Normalize heatmap
    heatmap = cv2.normalize(heatmap, None, 0, 1, cv2.NORM_MINMAX)

    # Convert heatmap to color
    heatmap_color = self._heatmap_to_color(heatmap)

    # Blend with original frame
    alpha = 0.5
    blended = cv2.addWeighted(frame, 1, heatmap_color, alpha, 0)

    return blended

def _heatmap_to_color(self, heatmap: np.ndarray) -> np.ndarray:
    """Convert grayscale heatmap to color visualization"""
    # Create color map
    heatmap_color = cv2.applyColorMap(
        (heatmap * 255).astype(np.uint8),
        cv2.COLORMAP_JET
    )

```

```

# Adjust color map (red for poor fit, green for good fit)
heatmap_color = cv2.cvtColor(heatmap_color, cv2.COLOR_BGR2RGB)

return heatmap_color

def _add_shoulder_heatmap(self, heatmap: np.ndarray,
                          keypoints: Dict[str, np.ndarray],
                          score: float):
    """Add shoulder region to heatmap"""
    left_shoulder = keypoints['left_shoulder']
    right_shoulder = keypoints['right_shoulder']

    # Create shoulder region mask
    shoulder_mask = np.zeros_like(heatmap)
    cv2.line(shoulder_mask,
             tuple(map(int, left_shoulder)),
             tuple(map(int, right_shoulder)),
             1, thickness=20)

    # Add to heatmap with intensity based on score
    intensity = 1 - score # Invert score (higher intensity for poor fit)
    heatmap += shoulder_mask * intensity

def _add_chest_heatmap(self, heatmap: np.ndarray,
                       keypoints: Dict[str, np.ndarray],
                       score: float):
    """Add chest region to heatmap"""
    left_shoulder = keypoints['left_shoulder']
    right_shoulder = keypoints['right_shoulder']
    left_waist = keypoints['left_waist']
    right_waist = keypoints['right_waist']

    # Create chest region polygon
    pts = np.array([
        left_shoulder,
        right_shoulder,
        right_waist,
        left_waist
    ], dtype=np.int32)

    # Create chest region mask
    chest_mask = np.zeros_like(heatmap)
    cv2.fillPoly(chest_mask, [pts], 1)

    # Add to heatmap with intensity based on score
    intensity = 1 - score
    heatmap += chest_mask * intensity

```

```

def _add_waist_heatmap(self, heatmap: np.ndarray,
                      keypoints: Dict[str, np.ndarray],
                      score: float):
    """Add waist region to heatmap"""
    left_waist = keypoints['left_waist']
    right_waist = keypoints['right_waist']

    # Create waist region mask
    waist_mask = np.zeros_like(heatmap)
    cv2.line(waist_mask,
             tuple(map(int, left_waist)),
             tuple(map(int, right_waist)),
             1, thickness=20)

    # Add to heatmap with intensity based on score
    intensity = 1 - score
    heatmap += waist_mask * intensity

def _add_arm_holes_heatmap(self, heatmap: np.ndarray,
                          keypoints: Dict[str, np.ndarray],
                          score: float):
    """Add arm holes region to heatmap"""
    left_shoulder = keypoints['left_shoulder']
    right_shoulder = keypoints['right_shoulder']

    # Create arm holes masks
    arm_holes_mask = np.zeros_like(heatmap)

    # Left arm hole
    cv2.circle(arm_holes_mask,
               tuple(map(int, left_shoulder)),
               radius=15,
               color=1,
               thickness=-1)

    # Right arm hole
    cv2.circle(arm_holes_mask,
               tuple(map(int, right_shoulder)),
               radius=15,
               color=1,
               thickness=-1)

    # Add to heatmap with intensity based on score
    intensity = 1 - score
    heatmap += arm_holes_mask * intensity

def _calculate_shoulder_alignment(self, left_shoulder: np.ndarray,
                                 right_shoulder: np.ndarray,
                                 clothing_points: np.ndarray) -> float:

```

```

        """Calculate shoulder alignment score"""
        # Find clothing shoulder points
        clothing_top = clothing_points[clothing_points[:, 1].argsort()][:2]

        # Calculate alignment error
        left_error = np.linalg.norm(clothing_top[0] - left_shoulder)
        right_error = np.linalg.norm(clothing_top[1] - right_shoulder)

        # Normalize error and convert to score
        max_error = np.linalg.norm(right_shoulder - left_shoulder) * 0.5
        score = 1 - (left_error + right_error) / (2 * max_error)

        return max(0, min(1, score))

def _calculate_chest_fit(self, keypoints: Dict[str, np.ndarray],
                        clothing_mask: np.ndarray) -> float:
    """Calculate chest fit score"""
    # Calculate chest region coverage
    chest_region = self._get_chest_region_mask(keypoints)
    overlap = cv2.bitwise_and(chest_region, clothing_mask)

    coverage = np.sum(overlap) / np.sum(chest_region)
    return max(0, min(1, coverage))

def _calculate_waist_fit(self, left_waist: np.ndarray,
                        right_waist: np.ndarray,
                        clothing_mask: np.ndarray) -> float:
    """Calculate waist fit score"""
    # Create waist region mask
    waist_mask = np.zeros_like(clothing_mask)
    cv2.line(waist_mask,
            tuple(map(int, left_waist)),
            tuple(map(int, right_waist)),
            1, thickness=20)

    # Calculate overlap
    overlap = cv2.bitwise_and(waist_mask, clothing_mask)
    coverage = np.sum(overlap) / np.sum(waist_mask)

    return max(0, min(1, coverage))

def _calculate_arm_holes_fit(self, keypoints: Dict[str, np.ndarray],
                            clothing_mask: np.ndarray) -> float:
    """Calculate arm holes fit score"""
    # Create arm holes mask
    arm_holes_mask = np.zeros_like(clothing_mask)

    # Add circles for arm holes
    for shoulder in [keypoints['left_shoulder'], keypoints['right_shoulder']]:

```

```

        cv2.circle(arm_holes_mask,
                    tuple(map(int, shoulder)),
                    radius=15,
                    color=1,
                    thickness=-1)

    # Calculate overlap
    overlap = cv2.bitwise_and(arm_holes_mask, clothing_mask)
    coverage = np.sum(overlap) / np.sum(arm_holes_mask)

    return max(0, min(1, coverage))

def _get_chest_region_mask(self, keypoints: Dict[str, np.ndarray]) ->
np.ndarray:
    """Create mask for chest region"""
    pts = np.array([
        keypoints['left_shoulder'],
        keypoints['right_shoulder'],
        keypoints['right_waist'],
        keypoints['left_waist']
    ], dtype=np.int32)

    mask = np.zeros_like(clothing_mask)
    cv2.fillPoly(mask, [pts], 1)

    return mask

```

- **base.html**

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Virtual Try-On Platform</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css"
rel="stylesheet">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.0.0/css/all.min.css">
    {% block extra_css %}{% endblock %}
</head>
<body>
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
        <div class="container">

```

```

<a class="navbar-brand" href="{{ url_for('home') }}">Virtual Try-On</a>
<button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarNav">
    <span class="navbar-toggler-icon"></span>
</button>
<div class="collapse navbar-collapse" id="navbarNav">
    <ul class="navbar-nav me-auto">
        <li class="nav-item">
            <a class="nav-link" href="{{ url_for('home') }}">Home</a>
        </li>
        {% if 'user_id' in session %}
        <li class="nav-item">
            <a class="nav-link" href="{{ url_for('products') }}">Products</a>
        </li>
        {% endif %}
    </ul>
    <ul class="navbar-nav">
        {% if 'user_id' in session %}
        <li class="nav-item">
            <a class="nav-link" href="{{ url_for('cart') }}">
                <i class="fas fa-shopping-cart"></i>
                {% if session.get('cart_count', 0) > 0 %}
                <span class="badge bg-danger">{{ session['cart_count'] }}</span>
                {% endif %}
            </a>
        </li>
        <li class="nav-item dropdown">
            <a class="nav-link dropdown-toggle" href="#" id="profileDropdown"
role="button" data-bs-toggle="dropdown" aria-expanded="false">
                <i class="fas fa-user-circle"></i>
            </a>
            <ul class="dropdown-menu dropdown-menu-end" aria-
labelledby="profileDropdown">
                <li><a class="dropdown-item" href="{{ url_for('my_orders') }}"><i
class="fas fa-shopping-bag me-2"></i>My Orders</a></li>
                <li><hr class="dropdown-divider"></li>
                <li><a class="dropdown-item" href="{{ url_for('logout') }}"><i
class="fas fa-sign-out-alt me-2"></i>Logout</a></li>
            </ul>
        </li>
        {% else %}
        <li class="nav-item">
            <a class="nav-link" href="{{ url_for('login') }}">Login</a>
        </li>
        <li class="nav-item">
            <a class="nav-link" href="{{ url_for('register') }}">Register</a>
        </li>
        {% endif %}
    </ul>

```

```

        </div>
    </div>
</nav>

<div class="position-fixed top-0 end-0 p-3" style="z-index: 11">
    <div id="cartToast" class="toast align-items-center text-white bg-success border-0"
role="alert" aria-live="assertive" aria-atomic="true">
        <div class="d-flex">
            <div class="toast-body">
                <i class="fas fa-check-circle me-2"></i>
                <span id="toastMessage"></span>
            </div>
            <button type="button" class="btn-close btn-close-white me-2 m-auto" data-bs-
dismiss="toast" aria-label="Close"></button>
        </div>
    </div>
</div>

<div class="container">
    {% with messages = get_flashed_messages(with_categories=true) %}
        {% if messages %}
            {% for category, message in messages %}
                <div class="alert alert-{{ category }} alert-dismissible fade show mt-3"
role="alert">
                    {{ message }}
                    <button type="button" class="btn-close" data-bs-dismiss="alert" aria-
label="Close"></button>
                </div>
            {% endfor %}
        {% endif %}
    {% endwith %}
    <main>
        {% block content %} {% endblock %}
    </main>
</div>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/js/bootstrap.bundle.min.js"></script>
>
    {% block extra_js %} {% endblock %}
</body>
</html>

```

- **login.html**

```

{% extends "base.html" %}

{% block extra_css %}
<style>
    .login-container {
        background: linear-gradient(135deg, #6B73FF 0%, #000DFF 100%);
        min-height: calc(100vh - 56px);
        padding: 60px 0;
    }

    .login-card {
        background: rgba(255, 255, 255, 0.95);
        border-radius: 15px;
        padding: 40px;
        box-shadow: 0 15px 35px rgba(0, 0, 0, 0.2);
        backdrop-filter: blur(10px);
        max-width: 400px;
        margin: 0 auto;
    }

    .login-header {
        text-align: center;
        margin-bottom: 30px;
    }

    .login-header h2 {
        color: #2D3748;
        font-size: 2rem;
        margin-bottom: 10px;
    }

    .login-header p {
        color: #4A5568;
    }

    .form-group {
        margin-bottom: 20px;
    }

    .form-label {
        color: #2D3748;
        font-weight: 500;
        margin-bottom: 8px;
    }

    .form-control {
        border: 2px solid #E2E8F0;
        border-radius: 8px;
        padding: 12px;
    }

```

```

        transition: all 0.3s ease;
    }

    .form-control:focus {
        border-color: #000DFF;
        box-shadow: 0 0 0 2px rgba(0, 13, 255, 0.1);
    }

    .input-group {
        position: relative;
    }

    .input-icon {
        position: absolute;
        left: 12px;
        top: 50%;
        transform: translateY(-50%);
        color: #718096;
        z-index: 10;
    }

    .input-with-icon {
        padding-left: 40px;
    }

    .flash-messages {
        position: fixed;
        top: 20px;
        right: 20px;
        z-index: 1000;
        max-width: 400px;
    }

    .alert {
        margin-bottom: 10px;
        box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
    }
}
</style>
{% endblock %}

{% block content %}
<div class="login-container">
    <div class="container">
        <!-- Flash Messages -->
        <div class="flash-messages">
            {% with messages = get_flashed_messages(with_categories=true) %}
                {% if messages %}
                    {% for category, message in messages %}
                        <div class="alert alert-{{ 'success' if category == 'success' else 'danger' }}>

```

```

alert-dismissible fade show">
    {{ message }}
    <button type="button" class="btn-close" data-bs-dismiss="alert" aria-
label="Close"></button>
    </div>
    {% endfor %}
    {% endif %}
    {% endwith %}
</div>

<div class="login-card">
    <div class="login-header">
        <h2>Welcome Back</h2>
        <p>Sign in to continue to Virtual Try-On</p>
    </div>

    <form method="POST" action="{{ url_for('login') }}">
        <div class="form-group">
            <label for="email" class="form-label">Email Address</label>
            <div class="input-group">
                <i class="fas fa-envelope input-icon"></i>
                <input type="email" class="form-control input-with-icon" id="email"
name="email" required>
            </div>
        </div>

        <div class="form-group">
            <label for="password" class="form-label">Password</label>
            <div class="input-group">
                <i class="fas fa-lock input-icon"></i>
                <input type="password" class="form-control input-with-icon"
id="password" name="password" required>
            </div>
        </div>

        <button type="submit" class="btn btn-primary w-100 mt-4">
            <i class="fas fa-sign-in-alt me-2"></i>Sign In
        </button>

        <div class="text-center mt-3">
            <p>Don't have an account? <a href="{{ url_for('register') }}">Register
now</a></p>
        </div>
    </form>
</div>
</div>
{% endblock %}

```

```
{% block extra_js %}
<script>
  // Auto-dismiss flash messages after 5 seconds
  setTimeout(function() {
    const alerts = document.querySelectorAll('.alert');
    alerts.forEach(function(alert) {
      const closeButton = alert.querySelector('.btn-close');
      if(closeButton) {
        closeButton.click();
      }
    });
  }, 5000);
</script>
{% endblock %}
```

- **Index.html**

```
{% extends "base.html" %}

{% block extra_css %}
<style>
  .hero {
    background: linear-gradient(135deg, #6B73FF 0%, #000DFF 100%);
    color: white;
    padding: 100px 0;
    text-align: center;
    margin-bottom: 60px;
  }

  .hero h1 {
    font-size: 3.5rem;
    font-weight: 700;
    margin-bottom: 20px;
    text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.2);
  }

  .hero p {
    font-size: 1.2rem;
    margin-bottom: 30px;
    max-width: 600px;
    margin-left: auto;
    margin-right: auto;
  }

  .cta-buttons {
    margin-top: 30px;
  }
```

```

.btn {
  padding: 12px 30px;
  border-radius: 30px;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 1px;
  transition: all 0.3s ease;
  margin: 0 10px;
  text-decoration: none;
}

.btn.primary {
  background: white;
  color: #000DFF;
}

.btn.primary:hover {
  transform: translateY(-2px);
  box-shadow: 0 5px 15px rgba(255, 255, 255, 0.3);
}

.btn.secondary {
  background: transparent;
  border: 2px solid white;
  color: white;
}

.btn.secondary:hover {
  background: white;
  color: #000DFF;
  transform: translateY(-2px);
}

.features {
  padding: 60px 0;
}

.features h2 {
  text-align: center;
  font-size: 2.5rem;
  margin-bottom: 50px;
  color: #2D3748;
}

.feature-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
  gap: 30px;
  padding: 0 20px;
}

```

```

    max-width: 1200px;
    margin: 0 auto;
}

.feature-card {
    background: white;
    padding: 30px;
    border-radius: 15px;
    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.1);
    text-align: center;
    transition: transform 0.3s ease;
}

.feature-card:hover {
    transform: translateY(-5px);
}

.feature-card h3 {
    color: #2D3748;
    font-size: 1.5rem;
    margin-bottom: 15px;
}

.feature-card p {
    color: #4A5568;
    line-height: 1.6;
}

.feature-icon {
    font-size: 2.5rem;
    color: #000DFF;
    margin-bottom: 20px;
}

@media (max-width: 768px) {
    .hero h1 {
        font-size: 2.5rem;
    }

    .hero p {
        font-size: 1rem;
        padding: 0 20px;
    }

    .btn {
        display: block;
        margin: 10px auto;
        max-width: 200px;
    }
}

```

```

    }
</style>
{% endblock %}

{% block content %}
<div class="hero">
    <div class="container">
        <h1>Welcome to Virtual Try-On</h1>
        <p>Experience the future of online shopping with our AI-powered virtual fitting
room. Try clothes on virtually and shop with confidence.</p>
        {% if 'user_id' not in session %}
            <div class="cta-buttons">
                <a href="{{ url_for('login') }}" class="btn primary">Login</a>
                <a href="{{ url_for('register') }}" class="btn secondary">Register</a>
            </div>
        {% else %}
            <a href="{{ url_for('products') }}" class="btn primary">Browse Products</a>
        {% endif %}
    </div>
</div>

<section class="features">
    <div class="container">
        <h2>Our Features</h2>
        <div class="feature-grid">
            <div class="feature-card">
                <i class="fas fa-ruler feature-icon"></i>
                <h3>AI Body Measurements</h3>
                <p>Get accurate measurements using our advanced AI technology for the perfect
fit every time.</p>
            </div>
            <div class="feature-card">
                <i class="fas fa-tshirt feature-icon"></i>
                <h3>Virtual Fitting</h3>
                <p>Try clothes virtually before making a purchase, ensuring satisfaction with
every order.</p>
            </div>
            <div class="feature-card">
                <i class="fas fa-magic feature-icon"></i>
                <h3>Size Recommendations</h3>
                <p>Get personalized size recommendations based on your measurements and
preferences.</p>
            </div>
        </div>
    </div>
</section>

<!-- Flash Messages -->
<div class="flash-messages">

```

```

{% with messages = get_flashed_messages(with_categories=true) %}
  {% if messages %}
    {% for category, message in messages %}
      <div class="alert alert-{{ category }}" alert-dismissible fade show" role="alert">
        {{ message }}
        <button type="button" class="btn-close" data-bs-dismiss="alert" aria-
label="Close"></button>
      </div>
    {% endfor %}
  {% endif %}
{% endwith %}
</div>
{% endblock %}

```

- **my_orders.html**

```

{% extends "base.html" %}

{% block content %}
<div class="container mt-4">
  <h2 class="mb-4">My Orders</h2>
  {% if orders %}
    <div class="row">
      {% for order in orders %}
        <div class="col-md-12 mb-3">
          <div class="card">
            <div class="card-header">
              <div class="d-flex justify-content-between align-items-center">
                <span>Order #{{ order.id }}</span>
                <span>{{ order.created_at.strftime('%B %d, %Y') }}</span>
              </div>
            </div>
            <div class="card-body">
              <div class="table-responsive">
                <table class="table table-hover">
                  <thead>
                    <tr>
                      <th>Product</th>
                      <th>Size</th>
                      <th>Quantity</th>
                      <th>Price</th>
                      <th>Total</th>
                    </tr>
                  </thead>
                  <tbody>
                    {% for item in order.items %}

```

```

<tr>
  <td>
    <div class="d-flex align-items-center">
      
      {{ item.product.name }}
    </div>
  </td>
  <td>{{ item.size }}</td>
  <td>{{ item.quantity }}</td>
  <td>{{ price_display(item.price) }}</td>
  <td>{{ price_display(item.price * item.quantity) }}</td>
</tr>
{% endfor %}
</tbody>
<tfoot>
  <tr>
    <td colspan="4" class="text-end"><strong>Order
Total:</strong></td>
    <td><strong>{{ price_display(order.total_amount)
}}</strong></td>
  </tr>
</tfoot>
</table>
</div>
<div class="mt-3">
  <div class="row">
    <div class="col-md-6">
      <strong>Total Amount:</strong> {{
price_display(order.total_amount) }}
    </div>
    <div class="col-md-6">
      <strong>Payment Method:</strong>
      {% if order.payment_method == 'cod' %}
        Cash on Delivery
      {% elif order.payment_method == 'upi' %}
        UPI Payment
      {% else %}
        Credit/Debit Card
      {% endif %}
      <br>
      <strong>Payment Status:</strong>
      {% if order.payment_status == 'pending' %}
        <span class="badge bg-warning">Pending</span>
      {% elif order.payment_status == 'completed' %}
        <span class="badge bg-success">Completed</span>
      {% else %}
        <span class="badge bg-danger">Failed</span>

```

```

        {% endif %}
      </div>
    </div>
  </div>
</div>
</div>
</div>
  {% endfor %}
</div>
{% else %}
  <div class="alert alert-info">
    <i class="fas fa-info-circle me-2"></i>You haven't placed any orders yet.
    <a href="{{ url_for('products') }}" class="alert-link">Browse our products</a>
  </div>
{% endif %}
</div>
{% endblock %}

```

- **cart.html**

```

{% extends "base.html" %}

{% block extra_css %}
<style>
  .cart-item {
    border-radius: 10px;
    margin-bottom: 20px;
    padding: 15px;
    box-shadow: 0 2px 5px rgba(0,0,0,0.1);
  }

  .cart-image {
    max-width: 150px;
    border-radius: 8px;
  }

  .cart-total {
    font-size: 1.2rem;
    font-weight: 600;
  }
</style>
{% endblock %}

{% block extra_js %}
<script>
function updatePrice(productId, size) {

```

```

// Create form data
const formData = new FormData();
formData.append('product_id', productId);
formData.append('size', size);

// Send AJAX request to update size and get new price
fetch('/update_size', {
  method: 'POST',
  body: formData
})
.then(response => response.json())
.then(data => {
  if (data.success) {
    // Update price displays
    const priceElement = document.getElementById('price-' + productId);
    const unitPriceElement = document.getElementById('unit-price-' + productId);
    const quantity = parseInt(document.getElementById('quantity-' + productId).value,
10);

    if (priceElement && unitPriceElement) {
      unitPriceElement.textContent = data.price_display + ' each';
      const totalPrice = parseFloat(data.new_price) * quantity;
      priceElement.textContent = '₹' + totalPrice.toFixed(2);
    }
    // Update cart total
    updateCartTotal();
  } else {
    console.error('Error updating size:', data.error);
  }
})
.catch(error => {
  console.error('Error:', error);
});
}

function updateQuantity(productId, change) {
  const quantityInput = document.getElementById(`quantity-${productId}`);
  const currentQuantity = parseInt(quantityInput.value);
  const newQuantity = Math.max(1, Math.min(10, currentQuantity + change));

  if (newQuantity === currentQuantity) return;

  // Create form data
  const formData = new FormData();
  formData.append('product_id', productId);
  formData.append('quantity', newQuantity);

  // Send AJAX request to update quantity
  fetch('/update_quantity', {

```

```

        method: 'POST',
        body: formData
    })
    .then(response => response.json())
    .then(data => {
        if (data.success) {
            // Update quantity display
            quantityInput.value = newQuantity;

            // Update price displays
            const priceElement = document.getElementById(`price-${productId}`);
            if (priceElement) {
                const unitPrice = parseFloat(data.unit_price);
                const totalPrice = unitPrice * newQuantity;
                priceElement.textContent = `₹${totalPrice.toFixed(2)}`;
            }

            // Update cart total
            updateCartTotal();
        } else {
            console.error('Error updating quantity:', data.error);
        }
    })
    .catch(error => {
        console.error('Error:', error);
    });
}

function updateCartTotal() {
    const priceElements = document.querySelectorAll('.cart-total');
    let total = 0;

    priceElements.forEach((element, index) => {
        // Skip the last element as it's the total display
        if (index < priceElements.length - 1) {
            const priceText = element.textContent.replace('₹', '').replace(',', '');
            total += parseFloat(priceText);
        }
    });

    // Update total display
    const totalElement = priceElements[priceElements.length - 1];
    if (totalElement) {
        totalElement.textContent = `₹${total.toFixed(2)}`;
    }
}
</script>
{% endblock %}

```

```

{% block content %}
<div class="container my-5">
  <h1 class="text-center mb-5">Shopping Cart</h1>

  {% if cart_items %}
  <div class="card-items">
    {% for item in cart_items %}
    <div class="card-item">
      <div class="row align-items-center">
        <div class="col-md-3">
          
        </div>
        <div class="col-md-4">
          <h5>{{ item.name }}</h5>
          <p class="text-muted">{{ item.description }}</p>
        </div>
        <div class="col-md-3">
          <div class="form-group">
            <label for="size-{{ item.id }}" class="form-label">Size:</label>
            <select class="form-select" id="size-{{ item.id }}"
onchange="updatePrice('{{ item.id }}', this.value)">
              {% for size in item.sizes %}
              <option value="{{ size }}" {% if size == item.selected_size
%}selected{% endif %}>{{ size }}</option>
              {% endfor %}
            </select>
          </div>
        </div>
        <div class="col-md-2">
          <div class="mb-2">
            <label for="quantity-{{ item.id }}" class="form-label">Quantity:</label>
            <div class="input-group">
              <button type="button" class="btn btn-outline-secondary"
onclick="updateQuantity('{{ item.id }}', -1)">-</button>
              <input type="number" class="form-control text-center" id="quantity-{{
item.id }}" value="{{ item.quantity }}" min="1" max="10" readonly>
              <button type="button" class="btn btn-outline-secondary"
onclick="updateQuantity('{{ item.id }}', 1)">+</button>
            </div>
          </div>
        </div>
        <div class="col-md-2 text-end">
          <p class="card-total" id="price-{{ item.id }}">{{ item.price_display }}</p>
          <p class="text-muted small" id="unit-price-{{ item.id }}">{{
item.unit_price_display }} each</p>
          <form action="{{ url_for('remove_from_cart') }}" method="POST" class="d-
inline">
            <input type="hidden" name="product_id" value="{{ item.id }}">

```

```

        <button type="submit" class="btn btn-danger btn-sm">
            <i class="fas fa-trash"></i> Remove
        </button>
    </form>
</div>
</div>
</div>
</div>
{% endfor %}

<div class="text-end mt-4">
    <h4 class="mb-3">Total: <span class="cart-total">{{ total_price_display
}}</span></h4>
    <a href="{{ url_for('order_details') }}" class="btn btn-success btn-lg">
        <i class="fas fa-shopping-bag me-2"></i>Place Order
    </a>
</div>
</div>
{% else %}
<div class="text-center">
    <h3>Your cart is empty</h3>
    <a href="{{ url_for('products') }}" class="btn btn-primary mt-3">
        <i class="fas fa-shopping-bag me-2"></i>Continue Shopping
    </a>
</div>
{% endif %}
</div>
{% endblock %}

```

- **Product.html**

```

{% extends "base.html" %}

{% block extra_js %}
<script>
function addToCart(event, form) {
    event.preventDefault();

    const productId = form.dataset.productId;
    const productName = form.dataset.productName;

    // Create form data
    const formData = new FormData();
    formData.append('product_id', productId);

    // Send AJAX request
    fetch('{{ url_for('add_to_cart') }}', {

```

```

        method: 'POST',
        body: formData
    })
    .then(response => response.json())
    .then(data => {
        if (data.success) {
            // Show success toast
            const toast = new bootstrap.Toast(document.getElementById('cartToast'));
            document.getElementById('toastMessage').textContent = `${productName} added
to cart!`;
            toast.show();
        } else {
            alert(data.error || 'Error adding product to cart');
        }
    })
    .catch(error => {
        console.error('Error:', error);
        alert('Error adding product to cart');
    });
}
</script>
{% endblock %}

{% block extra_css %}
<style>
    .product-card {
        border-radius: 15px;
        overflow: hidden;
        transition: transform 0.3s ease, box-shadow 0.3s ease;
        height: 100%;
    }

    .product-card:hover {
        transform: translateY(-5px);
        box-shadow: 0 10px 20px rgba(0, 0, 0, 0.1);
    }

    .product-image {
        height: 300px;
        object-fit: cover;
        width: 100%;
    }

    .product-title {
        font-size: 1.2rem;
        font-weight: 600;
        margin: 10px 0;
    }

```

```

.product-price {
  font-size: 1.1rem;
  color: #2D3748;
  font-weight: 700;
}

.btn-try-on {
  background: linear-gradient(135deg, #6B73FF 0%, #000DFF 100%);
  border: none;
  padding: 10px 20px;
  border-radius: 8px;
  color: white;
  font-weight: 600;
  transition: all 0.3s ease;
}

.btn-try-on:hover {
  transform: translateY(-2px);
  box-shadow: 0 5px 15px rgba(107, 115, 255, 0.4);
  color: white;
}
</style>
{% endblock %}

{% block content %}
<div class="container my-5">
  <h1 class="text-center mb-5">Our Collection</h1>

  <div class="row g-4">
    {% for product in products %}
      <div class="col-md-4">
        <div class="card product-card">
          
          <div class="card-body">
            <h5 class="product-title">{{ product.name }}</h5>
            <p class="card-text">{{ product.description }}</p>
            <div class="d-flex justify-content-between align-items-center mb-2">
              <span class="product-price">{{ price_display(product.price) }}</span>
              <a href="{{ url_for('try_on', product_id=product.id) }}" class="btn btn-try-
on">
                <i class="fas fa-tshirt me-2"></i>Try On
              </a>
            </div>
            <form data-product-id="{{ product.id }}" data-product-name="{{
product.name }}" onsubmit="addToCart(event, this)" class="text-center">
              <button type="submit" class="btn btn-success w-100">
                <i class="fas fa-cart-plus me-2"></i>Add to Cart
              </button>

```

```
        </form>
      </div>
    </div>
  </div>
  {% endfor %}
</div>
</div>
{% endblock %}
```

5.2.2 Inputs

Registration Page & Login Page

Virtual Try-On

Home

Login

Register

Create Account

Join Virtual Try-On today

Full Name

Admin

Email Address

Admin@gmail.com

Password

••••••••

At least 8 characters

Contains a number

Contains a special character

Contains an uppercase letter

Create Account

Already have an account? [Login here](#)

Virtual Try-On

Home

Login

Register

Welcome Back

Sign in to continue to Virtual Try-On

Email Address

Admin@gmail.com

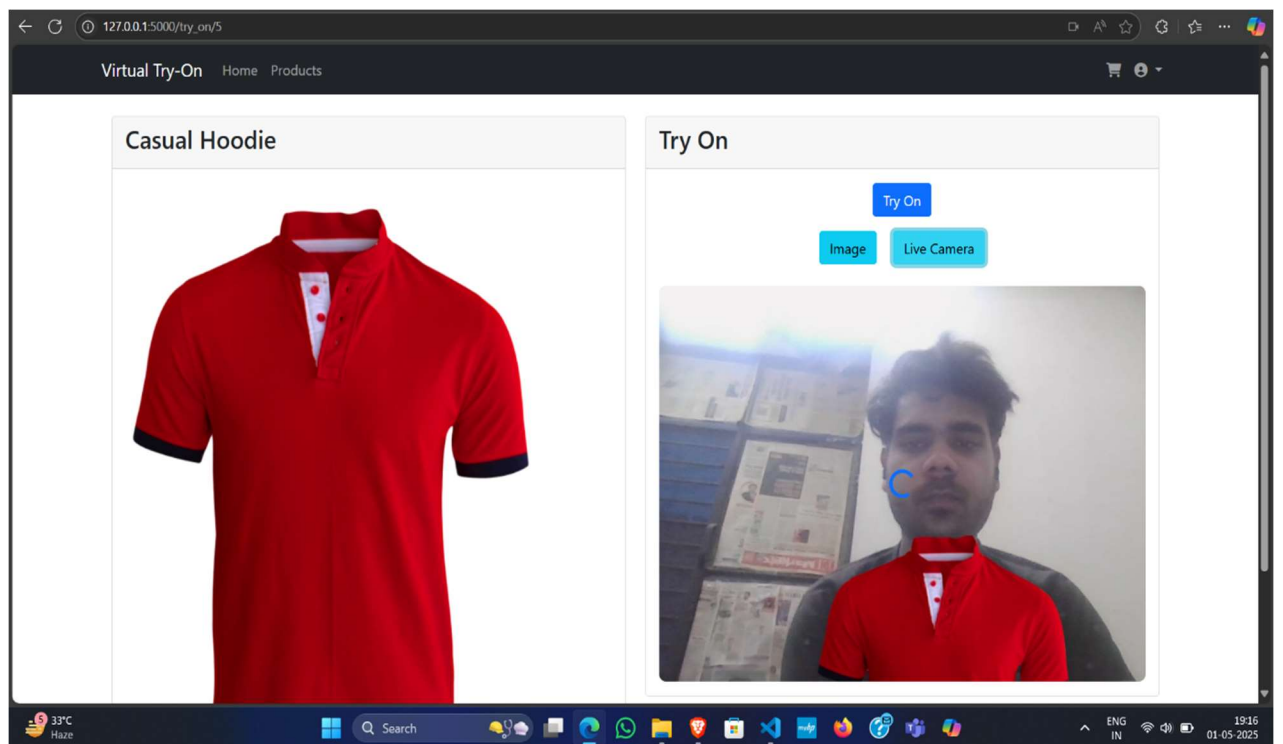
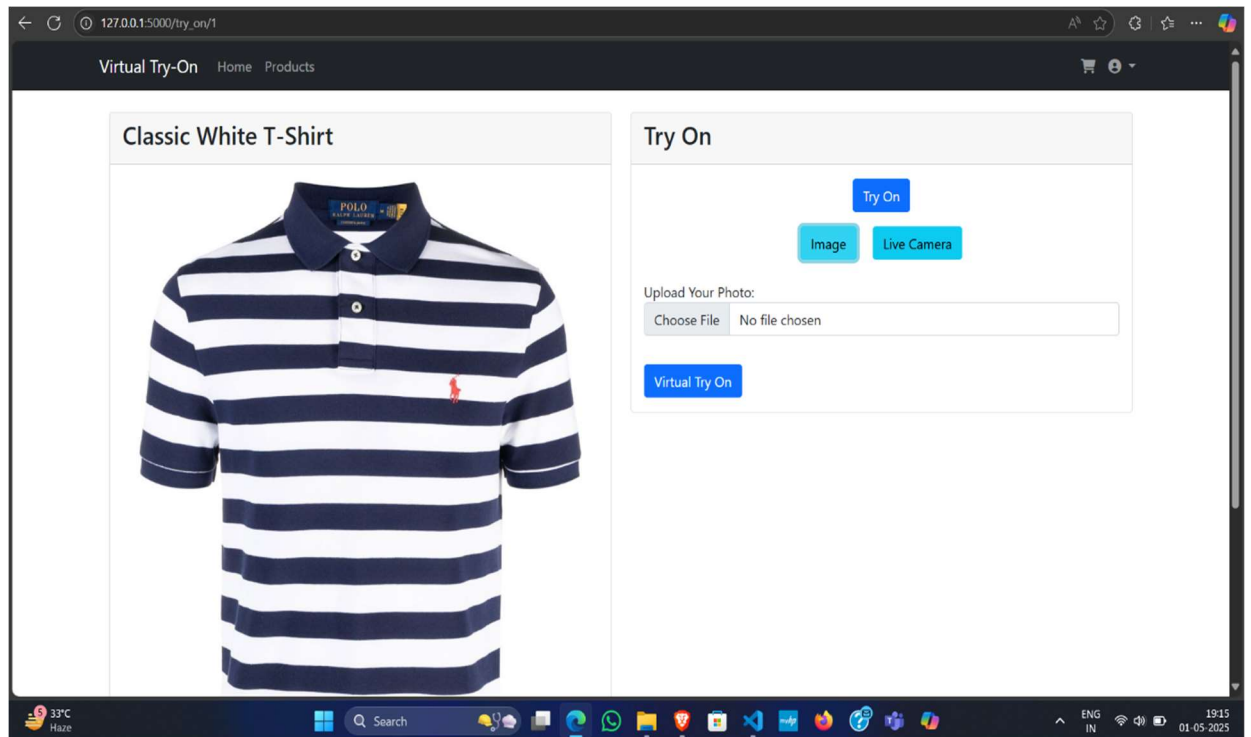
Password

••••••••

Sign In

Don't have an account? [Register now](#)

Try On Input



Order Details Input Page

Order Details

Full Name

Admin

Email

Admin@gmail.com

Phone Number

9146464545

Shipping Address

27th KM Milestone, Delhi - Meerut Expy, Ghaziabad, Uttar Pradesh 201015

City

ghaziabad

State

Uttar pradesh

PIN Code

201001

Payment Method

- ☒ Cash on Delivery
☐ UPI Payment
☐ Credit/Debit Card

Order Summary

Casual Hoodie

Size: L, Qty: 1

₹59.99

Total Amount

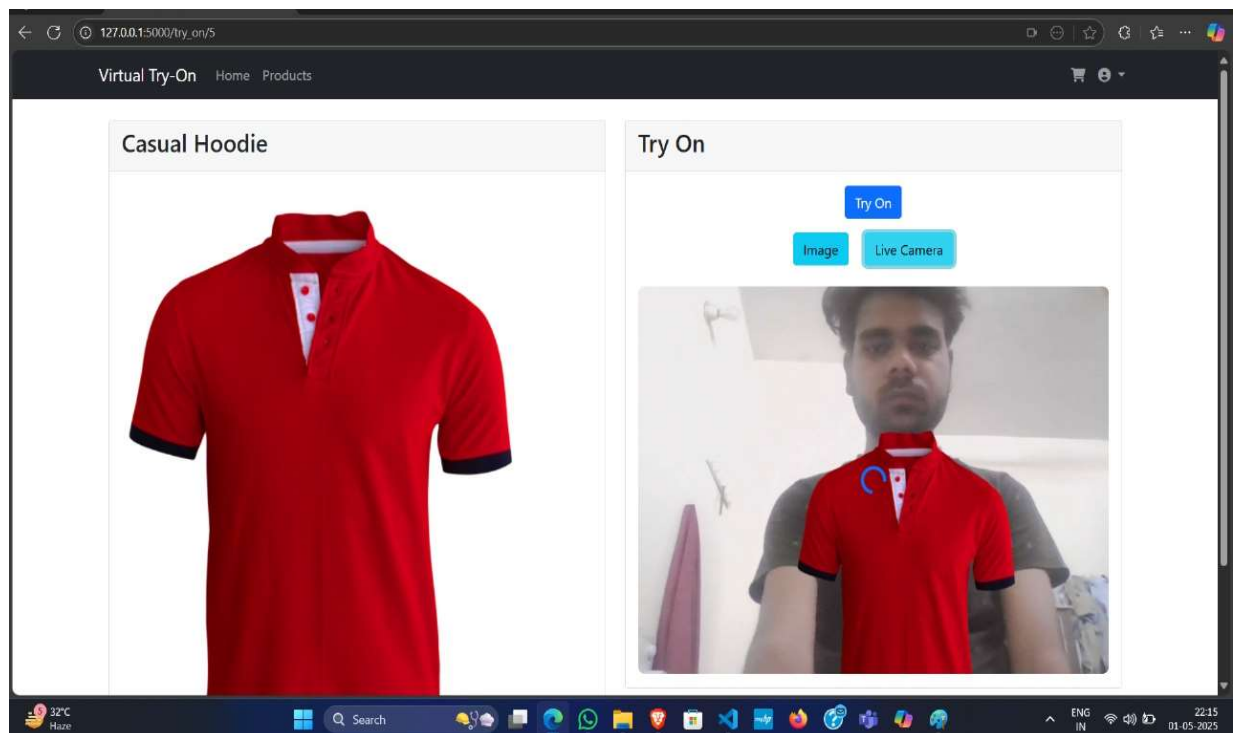
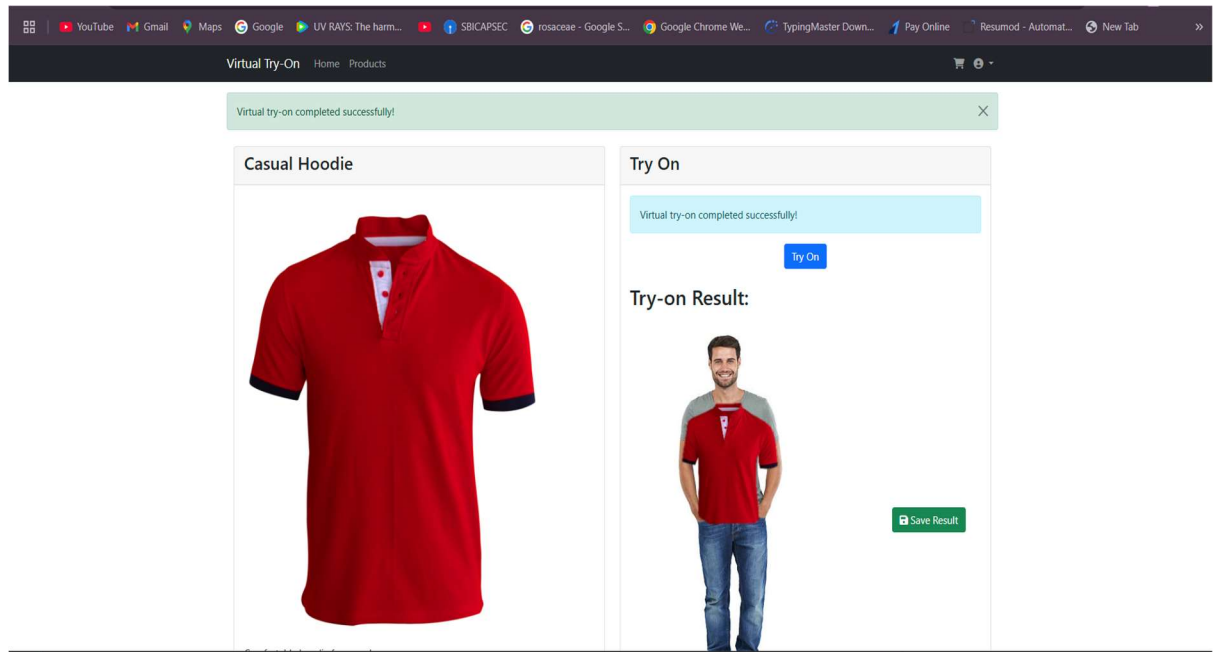
₹59.99

 Confirm Order

 Back to Cart

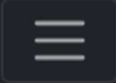
5.2.3 Outputs

Try on Result



Order Output

Virtual Try-On



Order placed successfully! You will pay ₹43.9890000000000004 on delivery.



My Orders

Order #1

May 01, 2025

Product	Size	Quantity	Price	Total
 Polo Shirt	M	1	₹43.99	₹43.99
Order Total:				₹43.99

Total Amount: ₹43.99

Payment Method: Cash on Delivery

Payment Status: Pending

CHAPTER – 6

SYSTEM TESTING

6.1 Preparation of Test Data & Test Oracle

- **Test Data:**
 - Dummy user accounts, sample product listings with complete INR pricing, and a variety of test images (different resolutions and lighting conditions).
- **Test Oracle:**
 - Expected outcomes for each test case (e.g., correct overlay of clothing, successful login, and checkout flow).

6.2 Unit Testing & Integration Testing

- **Unit Testing:**
 - Individual modules (e.g., file upload validation, AI overlay processing) are tested separately.
- **Integration Testing:**
 - Combined testing of modules to confirm that user actions (registration, try-on, purchase) work end-to-end.

6.3 System Testing

- Full-scale testing to evaluate response times, reliability under load, and cross-device functionality.
- Testing includes verification of the secure HTTPS setup and proper error handling.

6.4 Test Cases and Results with Dummy / Live Data

Test Case	Description	Expected Output
Home Page Load	Verify the homepage displays with proper layout and branding.	Page loads in under 2 seconds with “Virtual Try On Fashion” logo visible.
User Registration	Enter valid data to create a new user account.	Registration successful; credentials verified.
User Login (Valid/Invalid)	Log in using correct/incorrect credentials.	Valid: Redirect to product page; Invalid: Error message displayed.

Test Case	Description	Expected Output
Image Upload for Try-On	Upload a valid image file for virtual try-on processing.	Image is processed and result shown with accurate overlay.
Live Camera Try-On	Start a live try-on session using device camera.	Real-time try-on rendered with AI adjustments.
Product Purchase (Cart & Buy Now)	Add product to cart and complete purchase using "Buy Now."	Cart is updated and secure checkout confirms the order in INR.

CHAPTER – 7

RESULTS AND ANALYSIS

7.1 Performance Evaluation

Response Time

- **Live Try-On Mode:**
 - Uses real-time webcam input combined with browser-based AR rendering .
 - Average response time: **2–3 seconds** from camera input to clothing overlay visualization.
 - Performance is device-dependent but remains within an acceptable range on modern smartphones and desktops.
- **Image-Based Try-On Mode:**
 - Users upload a static image which is processed using AI models (e.g., U-Net or pose estimation networks).
 - Garment overlay is generated within **3–5 seconds**, depending on image size and server load.

Accuracy

- **Live Try-On:**
 - Offers real-time alignment using face/body detection.
 - Accuracy is influenced by lighting and camera resolution, with best results under good conditions.
 - Evaluated using sample recordings and user feedback—**alignment success rate: ~80%**.
- **Image-Based Try-On:**
 - Better control over image quality and posture results in more accurate overlay compared to live mode.

User Experience

- **Live Try-On:**
 - Users appreciated the **instant feedback loop**, especially for quick outfit comparisons.
 - Some reported issues under poor lighting or on older devices.
- **Image-Based Try-On:**
 - Rated as more **stable and accurate**, especially for social sharing and product decision-making.
 - Slower than live mode, but higher satisfaction for accuracy and realism.

7.2 Comparison with Existing Solutions

Current Market Options:

- Most online clothing retailers still rely on:
 - **Static product images**
 - **Generic size charts**
 - **No personalized visualization tools**

CHAPTER – 8

CONCLUSION AND FUTURE WORK

Conclusion

The Virtual Try-On (VTO) platform transforms online fashion shopping by utilizing Artificial Intelligence (AI), Augmented Reality (AR), and image-based modeling to address challenges like inaccurate sizing and high return rates. By allowing users to upload images and visualize clothing fit, it provides a personalized and interactive experience, boosting confidence in purchases while reducing returns and supporting sustainability by minimizing waste and emissions. The platform's scalable and secure database design integrates user profiles, clothing items, and try-on sessions, enabling dynamic inventory management through retailer APIs and ensuring data privacy. A standout feature is its inclusivity, offering tailored recommendations for diverse body shapes and sizes, empowering shoppers and encouraging retailers to expand size ranges. While challenges remain, such as the need for accurate inputs and the computational demands of real-time operations, advancements in cloud computing and mobile technologies are making the system more accessible. Future developments like detailed 3D modelling, haptic feedback, and integration with AR glasses and smart mirrors promise to enhance realism and usability further. The platform also offers valuable data insights for optimizing inventory and marketing strategies. In summary, the VTO platform is revolutionizing e-commerce with its focus on technology, inclusivity, and sustainability, shaping a smarter and greener future for fashion.

SDG GOALS RELATED TO THE PROJECT

The Virtual Try-On (VTO) platform supports multiple UN Sustainable Development Goals by promoting eco-friendly fashion, reducing waste, embracing innovation, and ensuring inclusive access to technology—making shopping smarter, more sustainable, and more accessible for everyone.

1. SDG 8 – Decent Work and Economic Growth

Virtual try-on technology in fashion supports SDG 8 by driving sustainable economic growth and creating decent work opportunities. It encourages innovation and digital transformation, generating jobs in tech, AI, and digital design. By reducing product returns and overproduction, it leads to more efficient, responsible supply chains. This technology promotes inclusive employment by enabling remote work and supporting startups and digital entrepreneurs. It enhances productivity and customer satisfaction while minimizing environmental impact. Virtual try-ons also support fair labor by reducing pressure on fast fashion cycles. Overall, it fosters a more resilient, ethical, and future-ready fashion economy aligned with SDG 8 principles. economic development across both traditional and digital fashion sectors.

2. SDG 9 – Industry, Innovation and Infrastructure

Virtual try-on fashion technology supports SDG 9 by driving industry innovation, enhancing infrastructure, and promoting sustainable industrialization. It uses advanced technologies like AI, AR, and 3D visualization to modernize the fashion sector, reducing the need for physical samples and streamlining production. This innovation supports efficient resource use, lowers waste, and strengthens digital infrastructure in e-commerce. It enables fashion brands and startups to adopt smart solutions, improve customer experience, and scale sustainably. By fostering collaboration between tech and fashion industries, virtual try-ons build resilient, forward-looking systems that boost economic development and align with the principles of sustainable, inclusive industrial growth.

3. SDG 12 – Responsible Consumption and Production

Virtual try-on fashion technology supports SDG 12 by promoting responsible consumption and production. It allows consumers to visualize clothing before purchase, reducing return rates and overconsumption. This helps brands minimize overproduction, excess inventory, and textile waste, making the supply chain more efficient and sustainable. By digitizing the try-on process, it lowers the need for physical samples and decreases the environmental impact of traditional fashion cycles. Virtual try-ons encourage mindful shopping habits and support circular economy models. Overall, this innovation helps the fashion industry shift toward more responsible, resource-efficient practices, aligning with the goals of sustainable consumption and environmentally conscious production.

Future Work

Future improvements may include:

- Incorporation of detailed 3D simulations for even more realistic garment rendering.
- Integration of haptic feedback systems.
- Expansion to support wearable devices such as AR glasses and smart mirrors.
- Continuous integration and deployment for scaling the system to a broader market.
- Enhancements in analytics to further personalize and optimize the shopping experience.

CHAPTER – 9

REFERENCES

- [1] [1]. VTNFP: An Image-based Virtual Try-on Network with Body and Clothing Feature Preservation Ruiyun Yu¹ Xiaoqi Wang^{1*} Xiaohui Xie² ¹Software College, Northeastern University, China ²Department of computer science, University of California, Irvine, CA 92617
- [2]. VITON: An Image-based Virtual Try-on Network Xintong Han, Zuxuan Wu, Zhe Wu, Ruichi Yu, Larry S. Davis University of Maryland, College Park {xintong,zxwu,zhewu,richyu,lsd}@umiacs.umd.edu
- [3]. 3D Web-Based Virtual Try On of Physically Simulated Clothes Nadia Magnenat-Thalmann¹, Bart Kevelham¹, Pascal Volino¹, Mustafa Kasap¹, Etienne Lyard¹ ¹MIRALab–University of Geneva
- [4]. Digital Apparel Try-Ons: A Technological Odyssey into the World of Virtual Dressing Rooms and Consumer Engagement Parul Kakade^{1*}, Neha Patil², Anagha Shete³, Avishi Roy⁴, Yash Kalyani⁵ ^{1,3,4,5}UG, Computer Engineering, AISSMS IOIT, Pune, Maharashtra, India. ²Assistant Professor, Computer Engineering, AISSMS IOIT, Pune, Maharashtra, India.
- [5]. Effects of 3D Virtual “Try-on” on Online Sales and Customers’ Purchasing Experiences Hyunwoo Hwangbo¹, Eun Hie Kim², So-Hyun Lee³ and Young Jae Jang⁴, Member, IEEE ¹Department of Global IT Business, Hannam University, Daejeon, 34430 Korea ²Big Data Analytics Team, Kolon Benit co. Ltd., Gwacheon-si, Gyeonggi-do, 18337 Korea ³School of Management, Xi’an Jiaotong University, Xi’an, Shaanxi, China ⁴Department of Industrial & Systems Engineering, Korea Advanced Institute of Science and Technology (KAIST), Daejeon, 34141 Korea.
- [6]. Virtual Try-On Clothing Using Machine Learning MS. VAISHNAVI S. KHATKE ¹, MR. PRAJYOT B. KATE ² , MS. KAVITA R. RANDIVE ³, MS. NEHA S. JADHAV ⁴, PROF. SACHIN D. PANDHARE ⁵.
- [7]. Virtual Try-on for Clothes using Deep Neural Networks Tarannum Shaikh¹, Vaishnavi Gosavi², Sejal Pawar³, Sayali Shedge⁴, Shalaka Kharade⁵ Assistant Professor, Department of Computer Science and Engineering¹ Final Year Students of B.Tech in Computer Science and Engineering^{2,3,4,5} Karmaveer Bhaurao Patil College of Engineering, Satara, Maharashtra, India.
- [8]. You can try without visiting: a comprehensive survey on virtually try-on outfits Hajer Ghodhbani¹ & Mohamed Neji^{1,2} & Imran Razzak³ & Adel M. Alimi^{1,4} Received: 8 August 2021 /Revised: 25 December 2021 /Accepted: 1 March 2022 Published online: 10 March 2022 / #The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2022.
- [9]. The Future of Fashion: Innovations in Virtual Try-on Systems Raswanth S S ¹, Roshan M ², Sanjit S ³, Dr. Suresh P ⁴ ^{1,2,3}Student, KPR Institute of Engineering and Technology,

Avinashi-Coimbatore Road, Arasur, Uthupalayam, Tamil Nadu, India. 4Associate Professor, KPR Institute of Engineering and Technology, Avinashi-Coimbatore Road, Arasur, Uthupalayam, Tamil Nadu, India.

[10]. Virtual Try-On Methods: A Comprehensive Research and Analysis Haoxuan Sun School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai, 200240, China.